

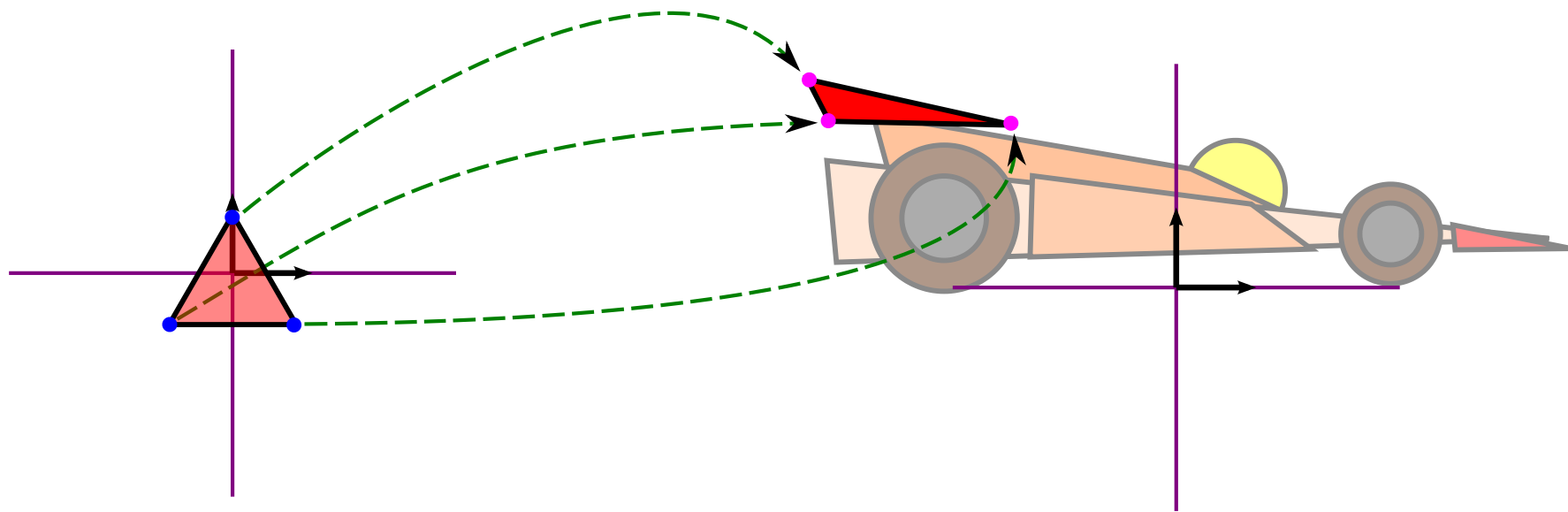
Espacios y Transformaciones

$$\begin{bmatrix} \cos 90^\circ & \sin 90^\circ \\ -\sin 90^\circ & \cos 90^\circ \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

DEFINICIÓN

- **Transformación:** función o mapeo que hace corresponder cada punto del espacio con otro punto del mismo espacio.

$$\hat{P} = T(P)$$



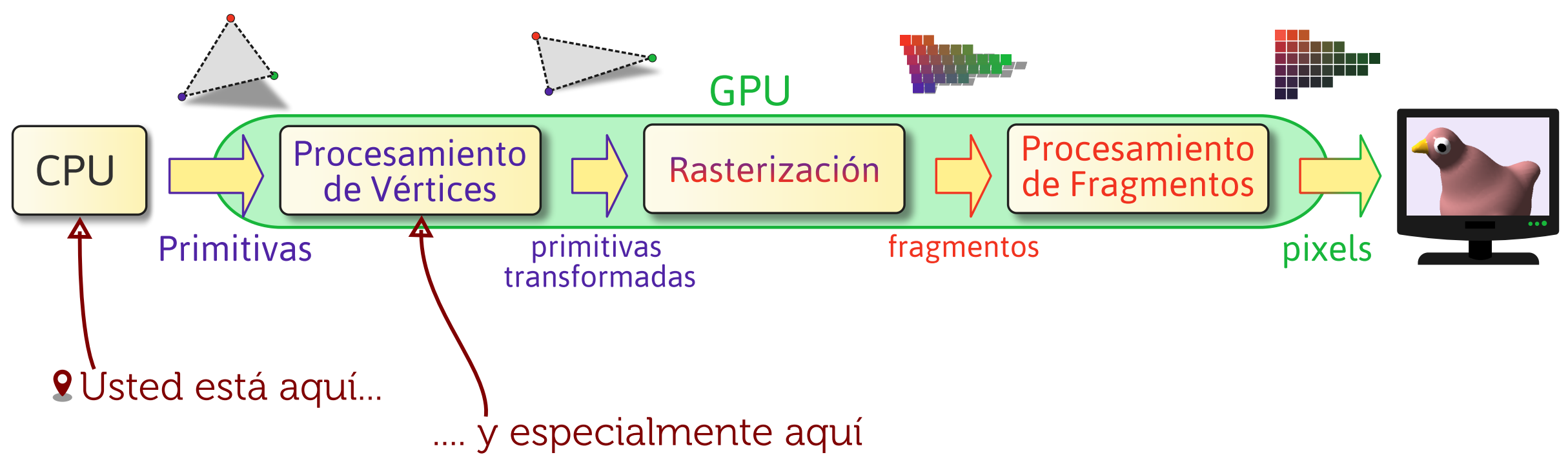
Una transformación en general es una operación que toma alguna entidad (punto, vector, color, etc) como entrada y lo convierte de alguna forma. En particular no interesará en esta unidad transformar puntos (posiciones de vértices). Podemos pensar en la transformación como una función que dado un punto (el original) nos retorna otro (el transformado).

Por ejemplo, si tenemos modelado el alerón del auto y debemos ubicarlo en relación al resto del auto, ese "ubicarlo" (que implica moverlo, rotarlo, escalarlo adecuadamente) es una transformación. Definido por vértices conectados formando primitivas, transformar todo el alerón es en realidad transformar cada uno de sus vértices (y formar las primitivas conectando los vértices transformados).

En general vamos a tener una serie de transformaciones para cada vértice. Por ahora puede resultar obvio que además de ubicar el alerón respecto del auto, también hay que mover el auto completo (alerón incluido) por la pista. Entonces, al alerón se le aplicarán al menos dos transformaciones. Pero hay varias más, aunque en principio no son tan obvias.

Se puede pensar así en una jerarquía de transformaciones. En el ejemplo todo el auto se mueve por la pista con una misma transformación (primer nivel, más general/arriba en la jerarquía), pero cada pieza tiene además una transformación propia (segundo nivel) para ensamblar correctamente el auto antes de moverlo.

USTED ESTÁ AQUÍ



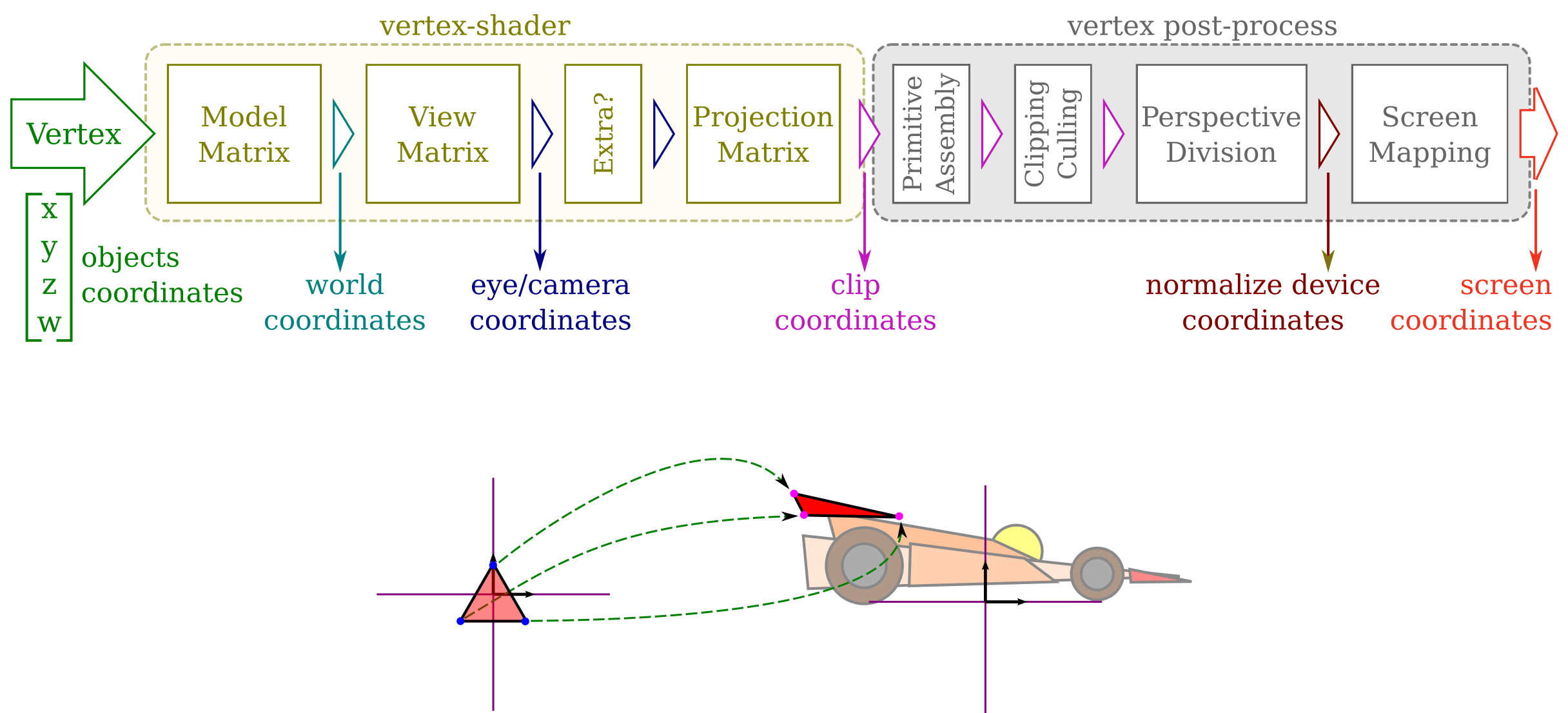
En el contexto de una aplicación gráfica, lo primero que nos interesa es transformar las primitivas para renderizarlas. En este caso, las transformaciones se aplicarán en la etapa de procesamiento por vértices. Pero en realidad, si bien ahí se aplican, las debe haber definido antes la aplicación principal y enviado luego a la GPU (como "uniform" del vertex shader). Entoces las transformaciones se definen y combinan en la aplicación, y se envían luego a la GPU para que las utilice con cada vértice.

Por razones que iremos descubriendo de a poco en esta teoría, representaremos a las transformaciones como matrices en un espacio de 4 dimensiones. Aplicar una transformación será multiplicar esa matriz por el punto a transformar. El programa principal en la CPU arma las matrices, y las envía a la GPU para que las aplique.

Esto es así porque las transformaciones varían mucho más que el modelo. Enviar los vértices transformados en cada frame sería enviar muchísima información. Cargar una sola vez los vértices originales a la GPU, e ir variando solo la matriz de transformación en cada frame es una forma muchísimo más eficiente de lograr las animaciones (al menos las que puedan representarse con esa matriz).

Además de la aplicación directa en el contexto de un algoritmo de renderizado, la aplicación también podría usar la misma teoría con otros fines.

MATRIZ MODELO

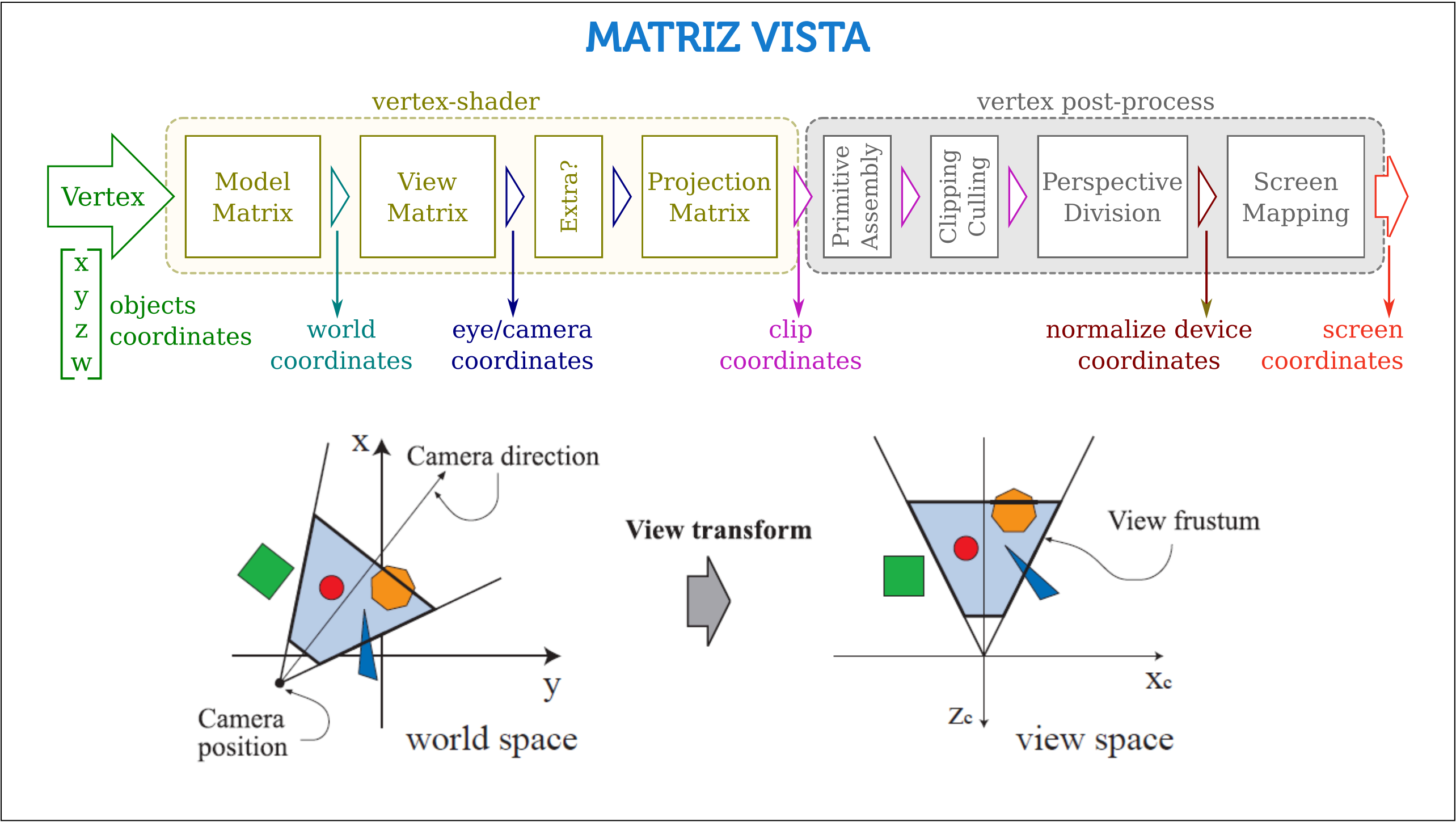


Ahora vamos a pasar a describir en más detalle lo que ocurre antes de la rasterización. La parte amarilla (vertex-shader) es la que tenemos que programar por completo. La parte gris (vertex post-process) es funcionalidad fija que resulta muy útil entender, pero que no queremos (ni podremos) variar demasiado (solo admite algunas mínimas configuraciones).

La matriz modelo es justamente la que mencionamos en el primer ejemplo, la más fácil de entender. El que diseña un "modelo" (ejs: una pieza del auto, un personaje de un juego, un arma o un árbol para el escenario), lo hace en un sistema de referencia arbitrario y de la forma que le resulte más cómodo para el diseño. Probablemente cada pieza esté centrada en o apoyada sobre el origen, pero la escala y la orientación son arbitrarias. Este es el **model space**, y varía para cada modelo.

Luego para ensamblar toda la escena, hay que definir un sistema de coordenadas único. Nuevamente es un sistema arbitrario que define a conveniencia el programador. En el ejemplo del auto (supongamos para simplificar que la pista es plana) seguramente se define este sistema de forma que el plano de la pista coincida con un plano del sistema (por ejemplo con el plano xy , $z=0$), y con el origen en algún punto fácil de identificar ($z=0$ xy en una esquina o en el centro del bounding-box). Es en este sistema, llamado **world space** (espacio del "mundo") donde hay que acomodar los modelos de la escena para que guarden la relación correcta entre sí.

Entonces el primer paso es pasar del model-space (coordenadas originales de los vértices) al world-space (su ubicación final en el sistema del "mundo" de la escena); y esto se hace con la matriz model (que por lo explicado anteriormente, puede ser resultado de combinar varias transformaciones de una jerarquía de modelos/transformaciones).



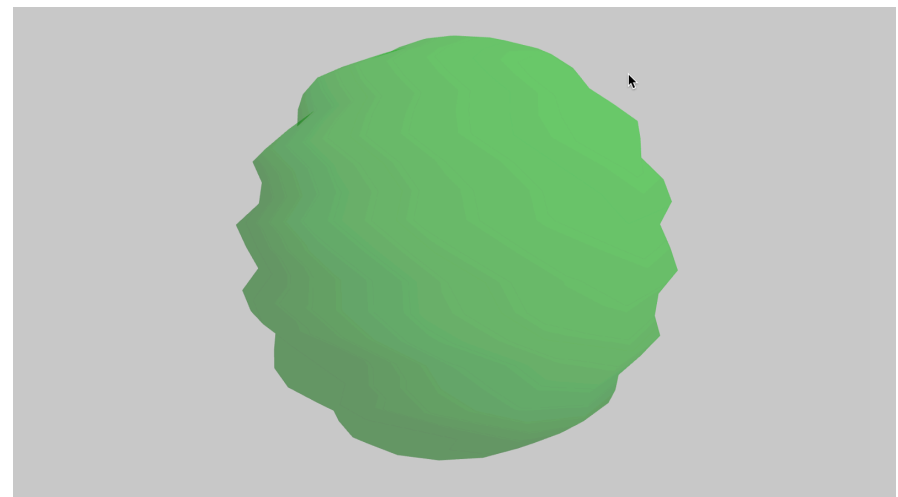
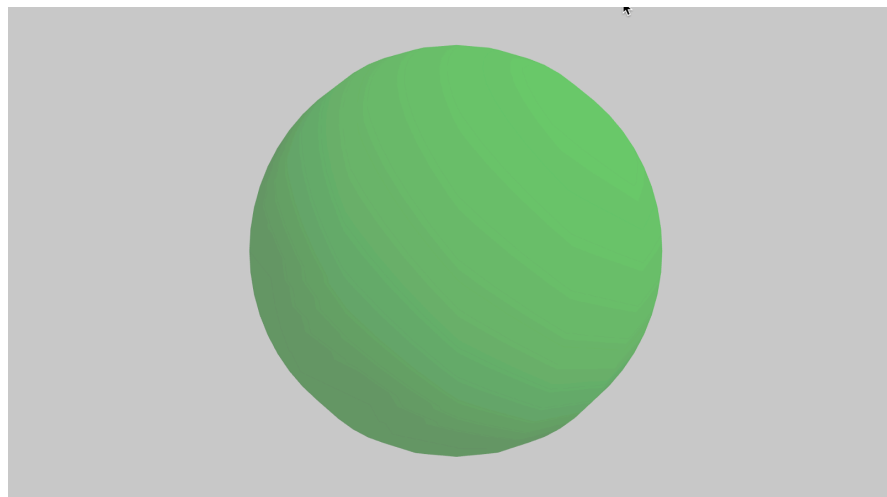
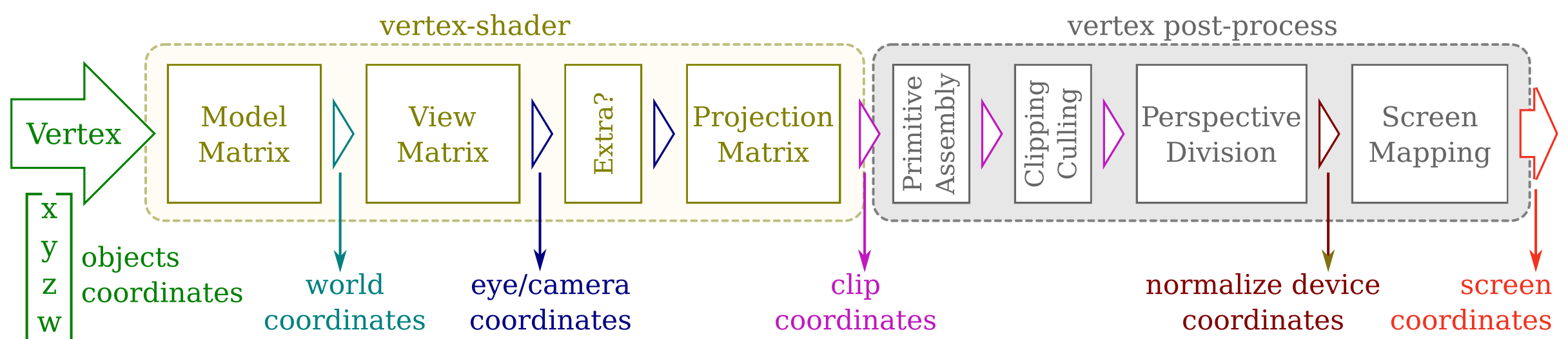
Hasta ahora hablamos de sistemas "arbitrarios" en el sentido de que son sistema que elige el usuario. A partir de aquí vamos pasando a sistemas de referencia definidos por la API (simpre tomaremos OpenGL como referencia, pero las ideas son las mismas en cualquier otra).

La **matriz view** transforma los vértices (que en este punto están en el sistema del mundo) al bold:sistema del ojo/de la cámara ((**eye/camera space**)). Por convención de OpenGL, la cámara siempre estará ubicada el el origen (punto 0,0,0), y mirando en dirección a -z (vector 0,0,-1). Si bien cuando usamos todo esto como caja negra solemos decir que con la matriz view ubicamos la cámara, técnicamente lo que ocurre es justo lo contrario.

La cámara no se mueve, siempre está en el origen; lo que se mueve es el mundo. Como lo que importan son las posiciones de los vértices "en relación a la cámara", entonces da lo mismo si la cámara avanza por ej 10 unidades en x, o si todo el mundo retrocede 10 unidades en x, la relación entre los vértices y la cámara será la misma en ambos casos.

Entonces, para "ubicar" la cámara, en realidad se define una matriz para mover al mundo (todos los vértices), no a la cámara, pero que logra el mismo efecto.

PASOS EXTRA

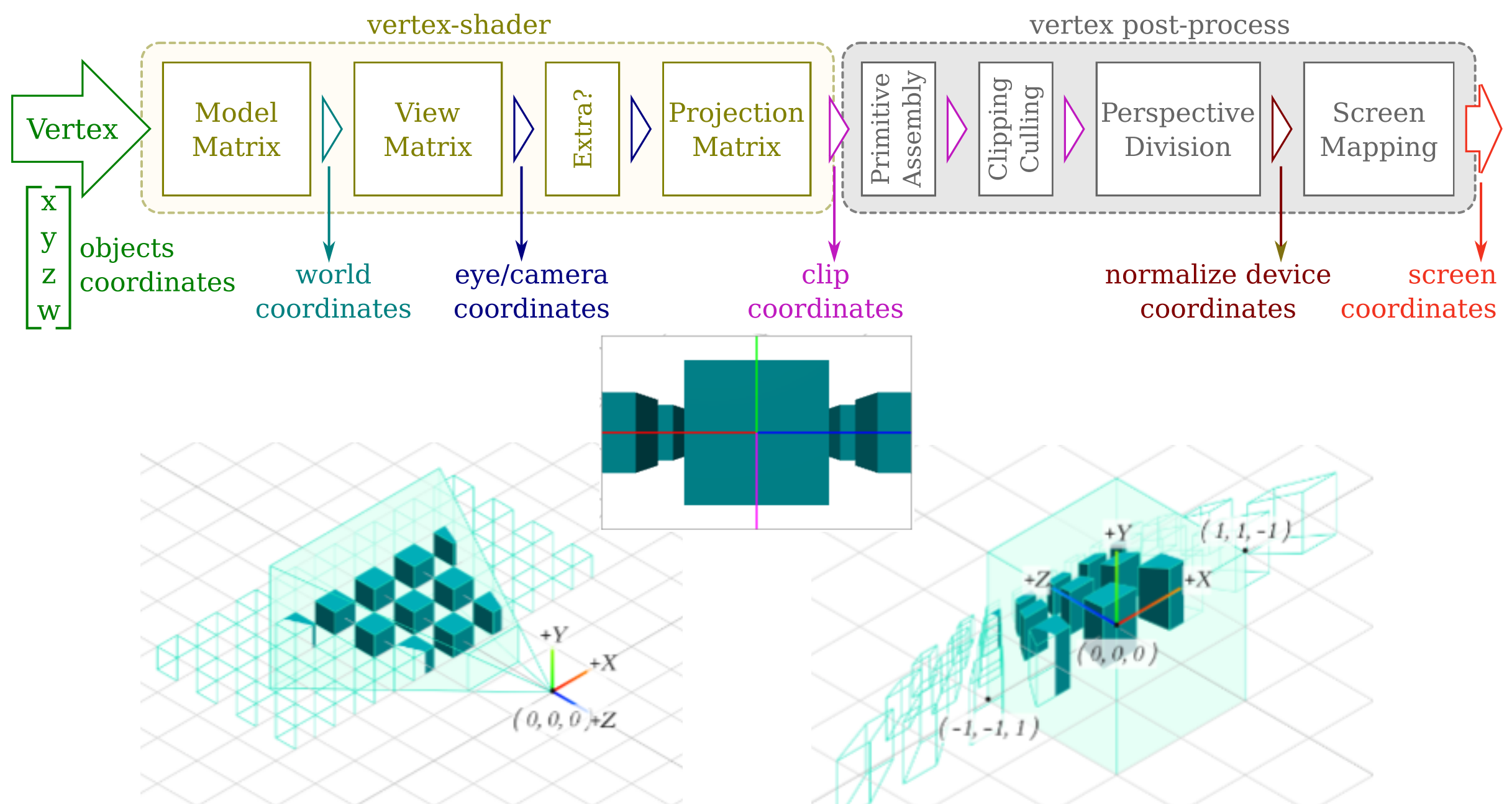


Ahora estamos en el espacio del ojo, y es el último espacio "normal" en algún sentido. Hasta aquí venimos haciendo transformaciones que no deforman demasiado los objetos, o que no "rompen" el sistema de referencia (veremos más adelante qué implica, pero hasta aquí son transformaciones "afines", la que sigue será "proyectiva"). Por eso, si queremos hacer alguna operación extra que modifique los vértices, este sería un buen lugar. Luego será más complicado por las transformaciones que vienen. Antes tal vez no convenga, porque en realidad a las matrices vista y modelo se las suele combinar en una sola para reducir el trabajo del shader. Entonces el primer paso sería aplicar la matriz "model-view", y el 2do (opcional) hacer algún truco o deformación adicional.

Ejemplos: en la image, es agregar algún "ruido" a los vértices para deformar la esfera; en el tp de iluminación era "inflar" el modelo para lograr el contorno.

Generalmente no haremos nada aquí (en los tps de la materia), pero podríamos. Dado que esta etapa "vertex-shader" es totalmente programable, en realidad no hace falta seguir este modelo. Pero se sigue igual, casi siempre el vertex shader se plantea a partir de las matrices que estamos mencionando, y en ese esquema cualquier "extra" iría seguramente en este paso.

MATRIZ PROYECCIÓN



La última transformación que vamos a aplicar en el vertex shader es la que llamamos **projection matrix**. No es en verdad un proyección (no perdemos aquí una dimensión todavía), pero sí es la que determinará el tipo de proyección que veremos en la imagen final.

La matriz de proyección define el volumen visual (qué se ve y qué no). Hasta ahora con la matriz vista ubicamos y alineamos la cámara, pero eso no alcanza para definir qué se ve.

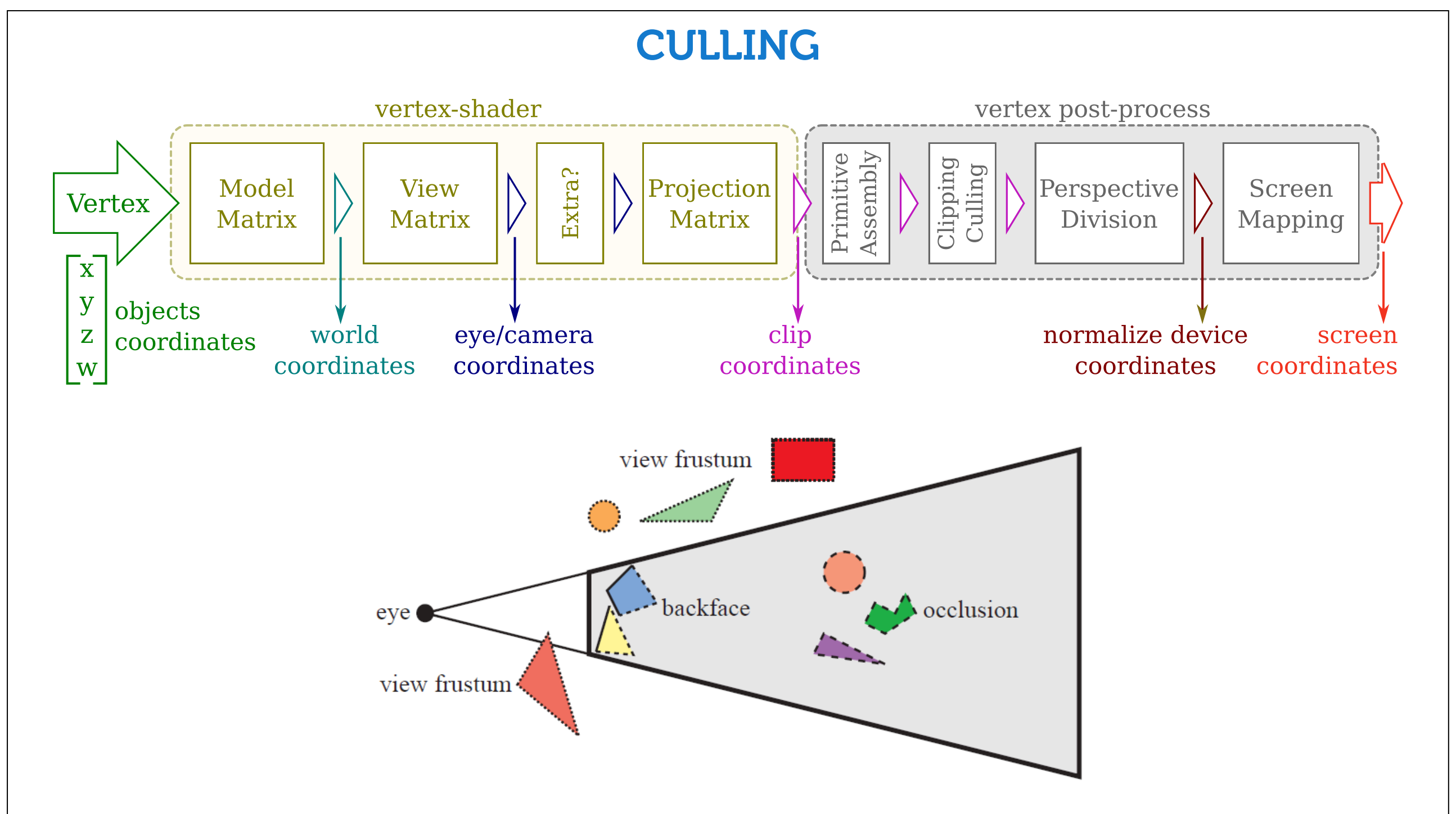
- ¿Cuánto abarca la imagen hace "arriba" y hacia los "costados"? No es lo mismo tomar un foto con un gran angular que con un lente macro.

- ¿Cuán cerca/lejos ve? Aquí la respuesta es más artificial y menos intuitiva por ahora. En la realidad, una cámara o un ojo verían potencialmente desde su posición hasta el infinito en la dirección en la que miran. En el pipeline de rasterización se definen límites (el por qué de estos límites se estudia en la unidad de buffers, tiene que ver con cómo se resuelven luego las oclusiones, el algoritmo del z-buffer).

Lo habitual para un efecto realista es quere usar proyección perspectiva. En esta proyección el volumen visual real sería como una pirámido (con vértice en el ojo). Pero el de OpenGL es una versión truncada de dicha pirámide, y a esa forma se la llama "frustum".

La matriz de proyección transforma el frustrum en un cubo centrado en el origen cuyos límites están en -1 y +1 en todas las direcciones (en medio cubo en realidad, los objetos están todos del mismo lado respecto a $z=0$). Observar en la figura como esta trasformación deforma los objetos generando el efecto de la vista en perspectiva: lo que están más atrás se aplastan/achican. La base del frustrum (z_{far} , el más alejado) es más grande su cara opuesta (z_{near} , el plano cercano al ojo), pero al mapearlas a este "cubo" ambas terminan del mismo tamaño, de ahí que lo que está más lejos (más cerca de z_{far}) se "aplasta" más que lo que está más cerca.

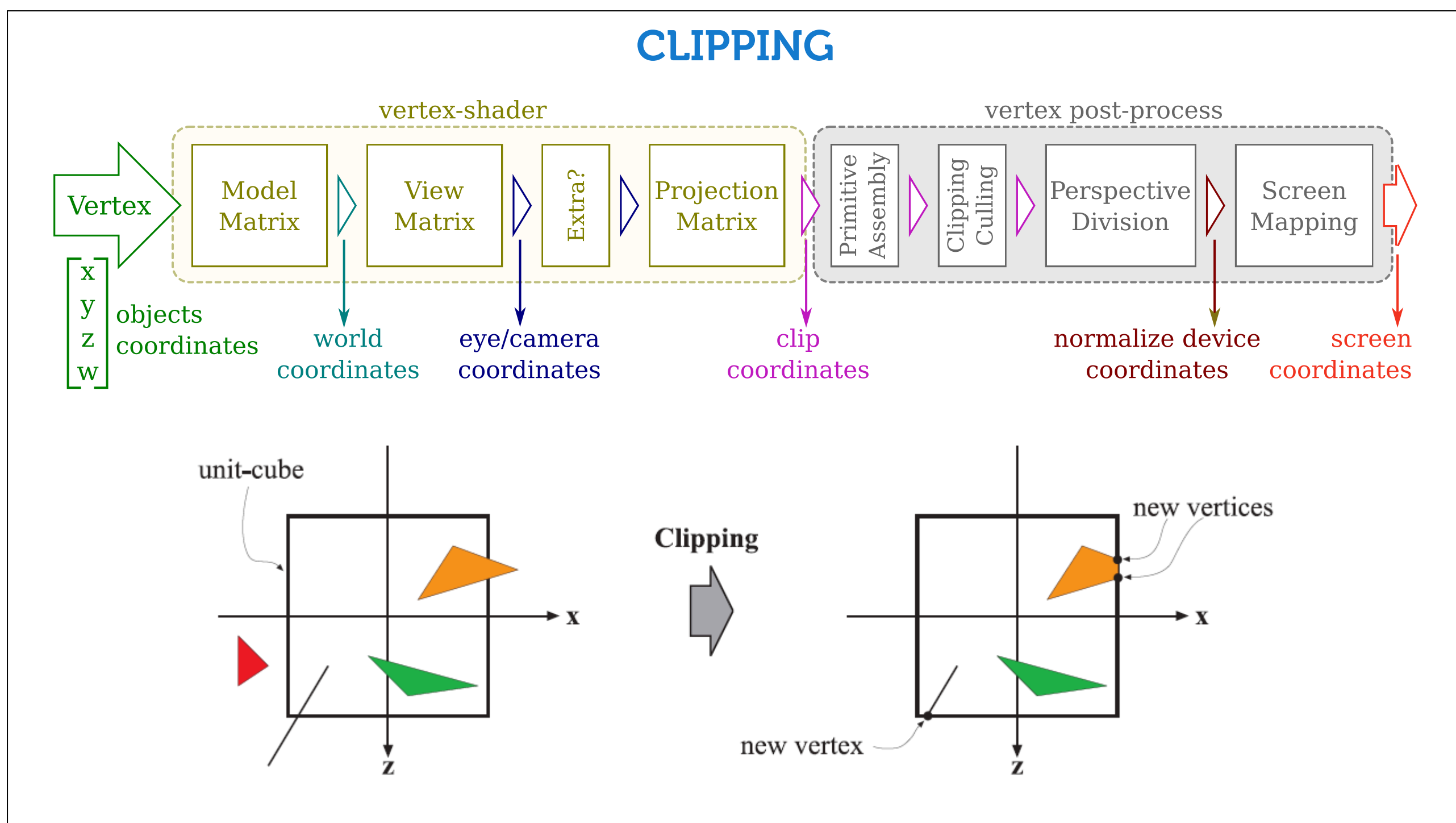
Para que esto resultara en una verdadera proyección, aquí habría que aplanar todo en alguna dirección (tirar z básicamente y quedarse con xy); pero quedan algunos pasos que conviene hacer antes de "peder" z , así que todavía no se hace.



Finalizado el vertex-shader (es decir, una vez transformados todos los vértices) hay que conectarlo para formar los triángulos (esto es la etapa de **primitive assembly**).

Luego se realiza el **clipping y el culling**. "Cull" es "descartar", "clip" es "recortar". La idea es quedarse solo con lo que vayamos a rasterizar, descartando todo lo que quedó fuera del volumen visual (frustum culling), y eventualmente las caras que no "miran" a la cámara (backface-culling). Además se recortan las primitivas que hayan quedado parte dentro y parte fuera, para quedarnos solo con la parte que se ve (clipping).

La figura muestra el frustum para entender mejor el propósito; pero en esta etapa el frustum ya se convirtió en un cubo en realidad. La figura menciona además el "occlusion" culling (modelos que no se ven porque otro los tapa), pero este no se realiza en el pipeline (el pipeline procesa a las primitivas individualmente, nunca podría "darse cuenta" de que un modelo tapa a otro... a esto se lo debe implementar en cpu).

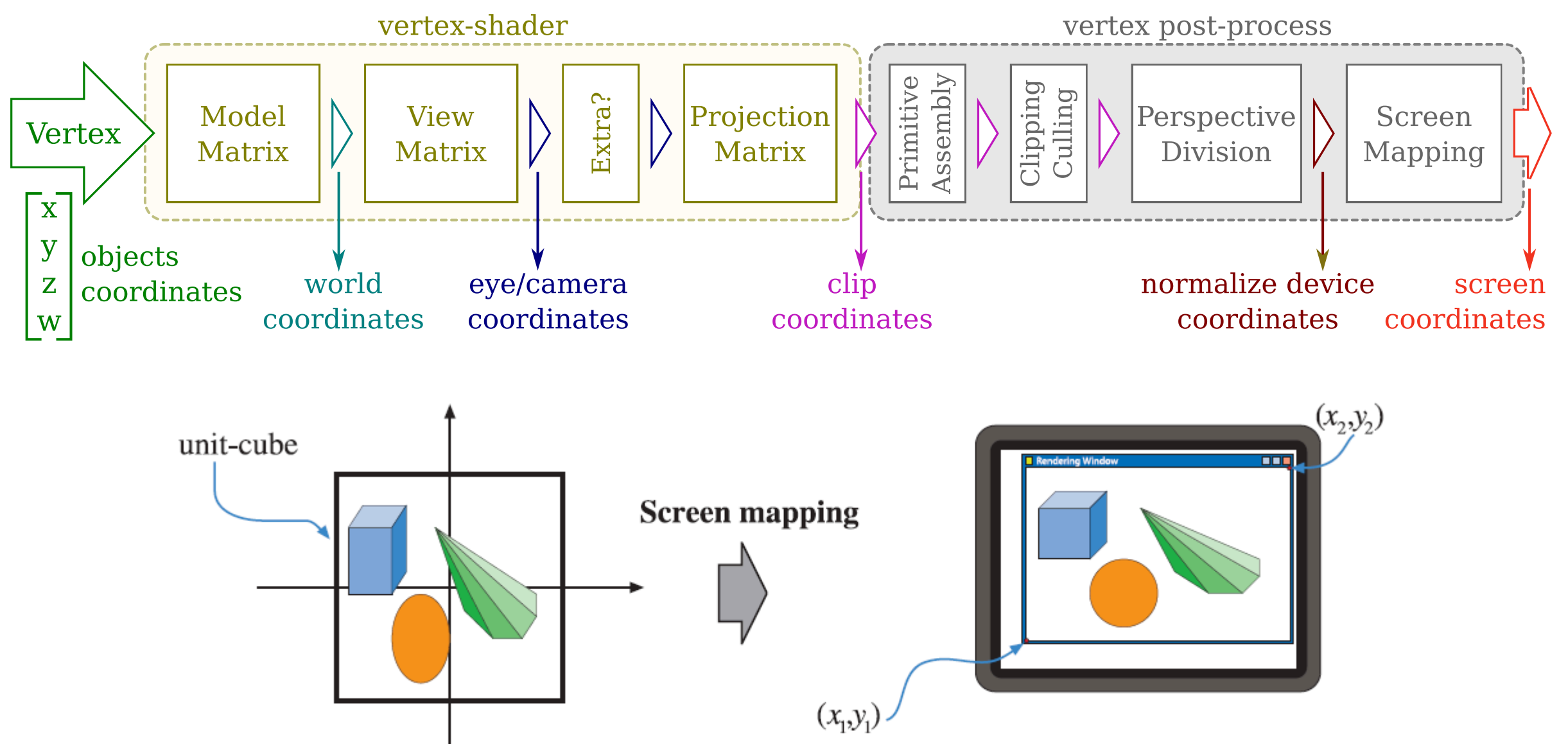


Clipping y culling ocurren luego de transformar el frustum en un cubo. Así entonces los cálculos para el descarte son mucho más simples. Para saber si un vértice queda dentro o fuera del volumen visual basta con verificar que sus coordenadas (en valor absoluto) no sean mayores a 1 (en realidad hay una 4ta dimensión involucrada que hemos omitido mencionar, pero no cambia el hecho de que la verificación sea casi trivial), mientras que en el espacio del mundo habría requerido averiguar de qué lado de cada plano del frustrum está (por eso se definen de forma tan rígida estos espacios, cada transformación nos lleva a un espacio donde lo que sigue se puede calcular de forma más eficiente).

Una vez "etiquetado" cada vértice como "dentro" o "fuera", es trivial descartar los triángulo que estén totalmente fuera, e identificar los que están un poco de cada lado y hay que recortar. En la figura, el anaranjado se recorte, del recorte queda un quad, que en realidad será partido por una diagonal y procesado como dos nuevos triángulos.

Aplicar el back-face culling también es mucho más simple aquí. Si la cámara está alinendada con el eje z, solo hay que mirar el signo de la componente z de la normal (no hace falta calcular la normal de la primitiva completa).

SCREEN MAPPING



Finalmente quedan las etapas de "perspective division" y "screen mapping" (también mencionada en muchos lugares como "viewport transformation").

"Perspective division" consiste en deshacerse de la 4ta dimensión. Hemos ignorado el hecho en las slides anteriores para simplificar la explicación; pero no podríamos codificar en una matriz las transformaciones que se describieron si trabajamos en R^3 . Utilizaremos un espacio auxiliar para poder implementar eficientemente estos cálculos, y ese espacio tendrá una 4ta dimensión adicional (es un truco matemático para poder representar todo con matrices). Veremos los detalles a medida que avance esta teoría. Por el momento solo basta decir que esta etapa se deshace de esa dimensión adicional y vuelve a R^3 .

- Aquí es donde realmente quedamos en el **NDC (normalized device coordinates)**, el cubo de -1 a +1 en x, y, z . Pero a fines prácticos el "clipping space" es equivalente, así que por simplificar muchas veces diremos que la matriz de proyección nos lleva directamente al NDC, aunque falte el paso de la división perspectiva.

El último paso, "screen mapping" simplemente estira este nuevo espacio (que es de 2×2 , va de -1 a 1 en x e y) al tamaño que corresponda para la rasterización (que es el tamaño de la ventana o del viewport dentro de ella: por ejemplo, 800x600 píxeles).

TRANSFORMACIÓN LINEAL

Un transf. es **lineal** si preserva la combinación lineal:

$$T(\alpha \mathbf{P}_1 + \beta \mathbf{P}_2) = \alpha T(\mathbf{P}_1) + \beta T(\mathbf{P}_2) \quad \Rightarrow \quad \hat{\mathbf{P}} = \underline{\underline{\mathbf{M}}}\mathbf{P}$$

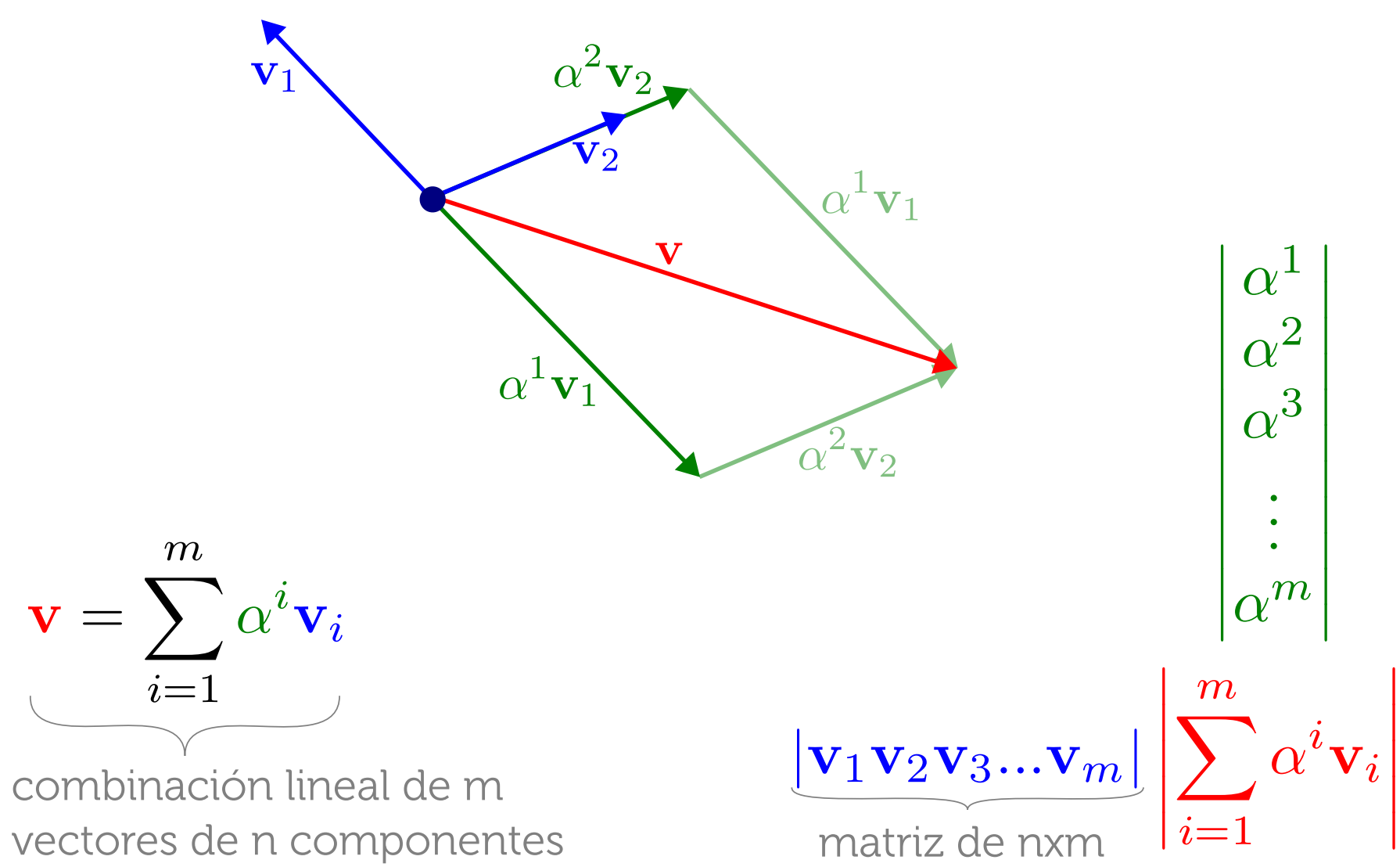
$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} Ax + By \\ Cx + Dy \end{bmatrix}$$

Entonces puede representarse como **matriz**!
Aplicar la transformación es premultiplicar por la matriz.

Una transformación es lineal si y solo si preserva la combinación lineal. Esto es, que aplicarle la transformación al punto resultante de una combinación lineal de m puntos, sea lo mismo que aplicarle primero la transformación a los m puntos y luego combinar linealmente (con los mismos pesos) el resultado.

Cuando una transformación es lineal se puede representar como una matriz, de forma que aplicarle la transformación a un punto sea simplemente un producto matriz-vector. Esto es algo muy eficiente de calcular, son solo unas pocas sumas y productos. Pero tiene además otras ventajas que iremos describiendo. Sin embargo, antes será importante entender cómo se "construye" la matriz de transformación.

ESPACIO VECTORIAL Y COMBINACIÓN LINEAL

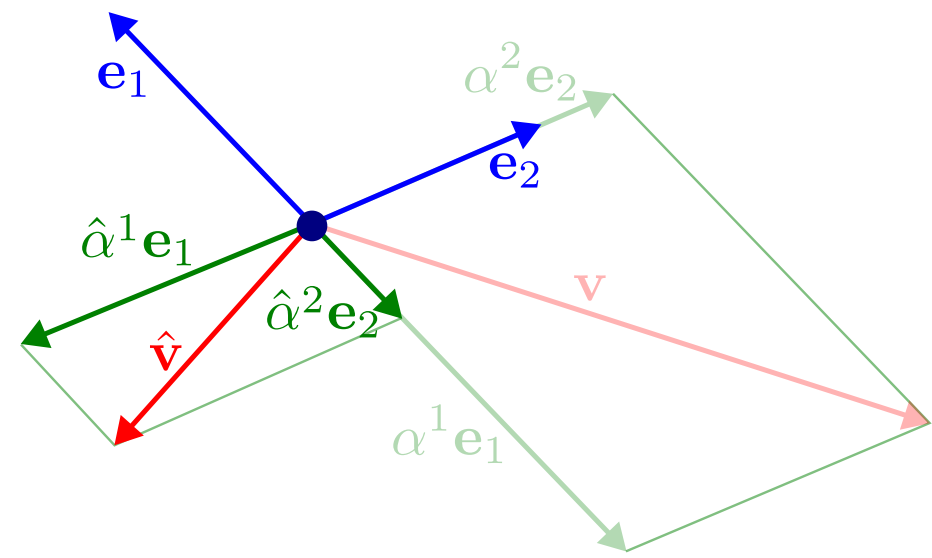
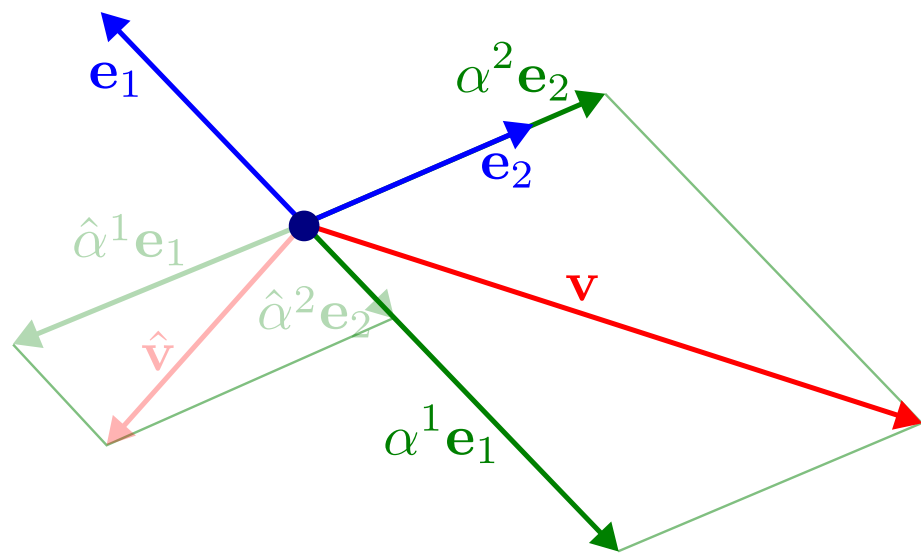


Estamos en un espacio n -dimensional, y tenemos un conjunto de m vectores $\mathbf{v}_i$ (los azules).

Puedo obtener un nuevo vector $\mathbf{v}$ (el rojo) como combinación lineal de otros $\mathbf{v}_i$ (o un punto p como el origen más una combinación lineal de esos vectores).

Notar que esta combinación lineal se puede plantear como una operación matricial: ponemos los $\mathbf{v}_i$ como columnas de la matriz y se la premultiplicamos a un vector formado por los pesos. El resultado será el vector $\mathbf{v}$.

ESPACIO VECTORIAL Y COMBINACIÓN LINEAL



$$\hat{\mathbf{v}} = \mathbf{L}(\mathbf{v}) = \mathbf{L}\left(\underbrace{\sum \alpha^i \mathbf{e}_i}_{\mathbf{v}}\right) = \sum \hat{\alpha}^i \mathbf{e}_i$$

Supongamos ahora que el conjunto de vectores forma una base. Es decir, son n vectores (para $\mathbb{R}^n$) y son L.I. (ninguno se puede obtener como combinación lineal de los otros). Pasamos a llamar a estos vectores bases $\mathbf{e}_i$.

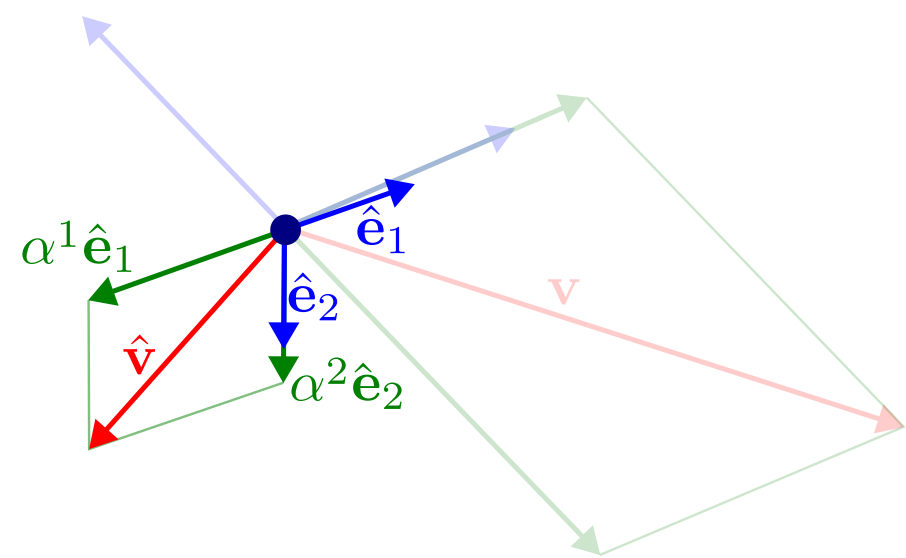
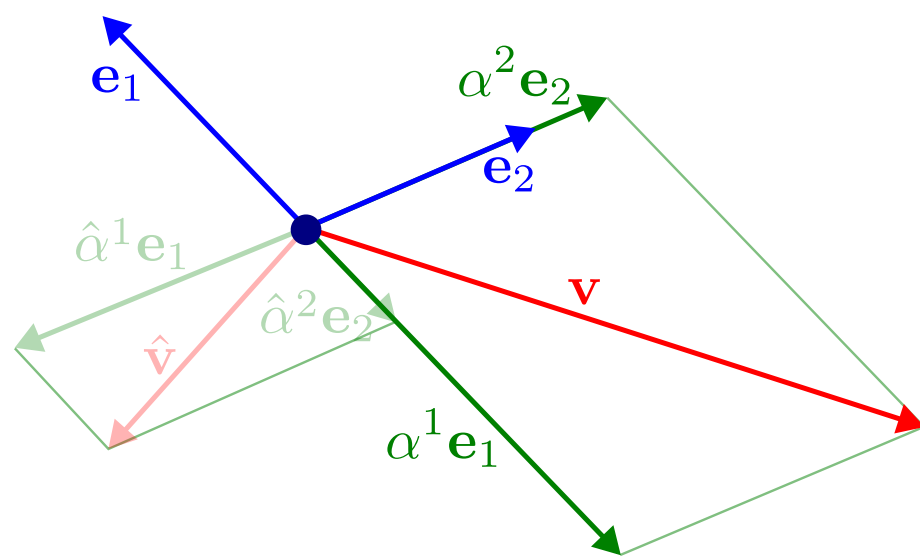
En la práctica casi siempre partiremos de la base canónica; pero lo que vamos a mostrar aplica para cualquier base (aunque no sean vectores unitarios ni ortogonales). Notar que un punto en un espacio está definido por sus coordenadas para una sistema de referencia (la base), que no son más que los pesos con los que combinar linealmente los vectores de la base para llegar desde el origen al punto.

Como ahora tenemos una base, cualquier vector de este espacio se puede obtener como combinación lineal de los de esa base.

En particular, la versión transformada de $\mathbf{v}$ se puede obtener como una nueva (otros pesos) combinación lineal.

Pero... Si la transformación es lineal, podemos sacar la sumatoria y los alphas afuera de $\mathbf{L}(\dots)$.

ESPACIO VECTORIAL Y COMBINACIÓN LINEAL



$$\hat{\mathbf{v}} = \mathbf{L}(\mathbf{v}) = \mathbf{L}\left(\underbrace{\sum \alpha^i \mathbf{e}_i}_{\mathbf{v}}\right) \overset{\text{por ser lineal}}{=} \sum \alpha^i \underbrace{\mathbf{L}(\mathbf{e}_i)}_{\hat{\mathbf{e}}_i} = \sum \alpha^i \hat{\mathbf{e}}_i$$

$$\begin{matrix} \hat{\mathbf{e}}_x \\ \hat{\mathbf{e}}_y \end{matrix} \begin{vmatrix} \hat{e}_x^y & \hat{e}_y^y \\ \hat{e}_x^x & \hat{e}_y^x \end{vmatrix} \begin{vmatrix} v^x \\ v^y \end{vmatrix} \begin{matrix} \hat{v}^x \\ \hat{v}^y \end{matrix}$$

Si sacamos la sumatoria y los alphas fuera de L, L queda transformando solo a los vectores bases, y entonces tenemos una combinación lineal de los vectores bases transformados, que utiliza los pesos originales.

El corolario de esto es que al final todo depende de lo que le pase a la base. Si conozco como se transforma la base, puedo ver fácilmente como se transforma cualquier cosa, ya que en la base transformada los coeficientes (las componentes del vector) serán los mismos que en la original.

Y recordemos que podemos representar esto como matriz usando los $\mathbf{e}_i$ como columnas. Entonces esto nos da una forma fácil de armar una matriz (si puedo medir o plantear los ejes transformados ya puedo armar la matriz); y de interpretar una matriz (identifico en ella los ejes transformados y así puedo "dibujar" la nueva base o deducir qué le hace esa transformación).

Esto último será muy importante en la práctica, porque nos permitirá plantear o interpretar matrices de transformación arbitrarias en un solo paso. Es decir, sin tener que descomponer la transformación en rotación+traslación+escalado+etc (ya que no siempre es obvia la combinación, y tampoco el orden, además de ser más trabajosa la composición/descomposición).

TRANSFORMACIÓN LINEAL

- Vector original:

$$\mathbf{v} = \sum v^i \mathbf{e}_i$$

- Componentes transformadas:

$$\hat{\mathbf{v}} = \sum \hat{v}^i \mathbf{e}_i$$

- Base transformada:

$$\hat{\mathbf{v}} = \sum v^i \hat{\mathbf{e}}_i$$

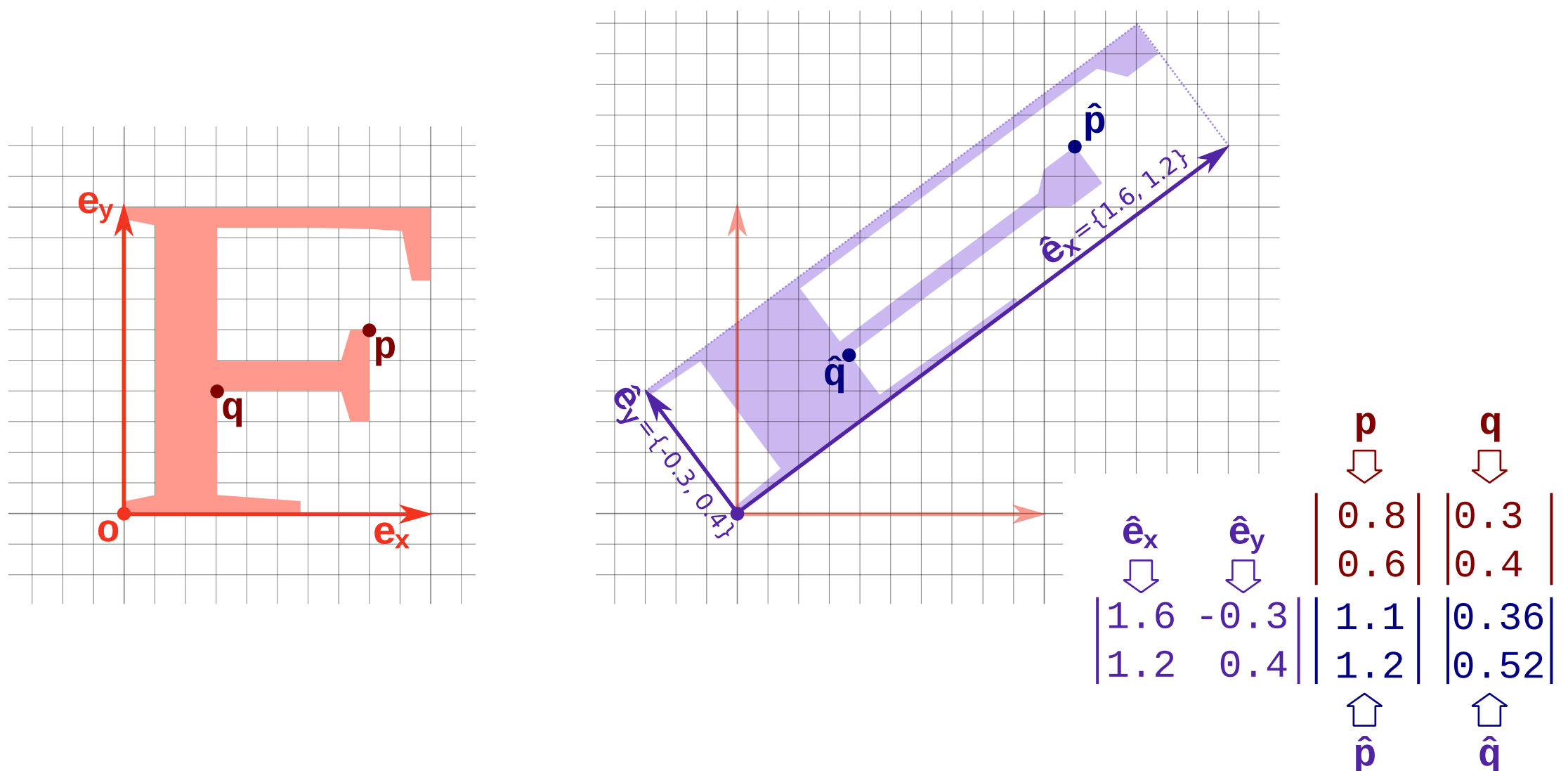
$$\underline{\underline{\mathbf{M}}} = \begin{bmatrix} \hat{e}_x^x & \hat{e}_x^y \\ \hat{e}_y^x & \hat{e}_y^y \end{bmatrix} \Rightarrow \hat{\mathbf{v}} = \underline{\underline{\mathbf{M}}} \mathbf{v}$$

Entonces tenemos dos formas de ver o pensar la transformación:

- Nuevos pesos para base original (donde "pesos" será en la práctica "componentes" de un vector o "coordenadas" de un punto).
- Nueva base para los mismos pesos. Esta segunda forma de verlo es más conveniente para interpretar o armar la matriz de transformación.

No es que queremos calcular la nueva base; queremos transformar un modelo, lo que implica transformar sus vértices y obtener las nuevas coordenadas (nuevos pesos: se corresponde con la 1ra forma de verlo). Pero para implementarlo eficientemente usaremos matrices, y para entender o plantear una matriz de transformación es más conveniente verlo de la 2da forma (si encuentro cómo quedarían los ejes transformados puedo armar fácilmente la matriz para luego aplicarsela a los vértices).

MATRIZ TRANSFORMACIÓN LINEAL



De aquí sale un ejercicio básico de examen (de los que no pueden no saber!). Hay dos presentaciones: dada una F transformada, armar la matriz; o dada una matriz, dibujar la F transformada.

En estos ejercicios el método correcto nunca es tratar de entender si se trata de una rotación, un escalado, un shear, o qué combinación en qué orden. Si bien teóricamente se podría, no solo es mucho más trabajo, sino que delata el hecho de no entender la relación entre las columnas de la matriz y los vectores base.

Si hay que armar la matriz: recordando que las columnas son las bases transformadas, todo lo que hay que hacer es identificar los vectores de la base transformada (lo cual debería ser más o menos obvio en relación a la F transformada), y medirlos (según la base original).

Si hay que "aplicar" la transformación: dibujamos la base transformada (las columnas de la matriz), y luego en referencia a ella la nueva F (es decir, replicamos las "proporciones" de la F original respecto a la base original, en la nueva F respecto a la nueva base).

En la PC, para transformar simplemente se van "pasando" los vértices [premultiplicando] por la matriz. El ejemplo muestra 2 vértices y sus transformados. Si en un ejercicio armaron una matriz, o dibujaron la F transformada y quieren después verificar si el resultado puede ser correcto, entonces pueden aplicar la matriz a dos o tres puntos/vectores y validar con el dibujo.

COMPOSICIÓN DE TRANSFORMACIONES

Aplicación de transformaciones sucesivas:

$$\mathbf{P}^1 = T^1(\mathbf{P})$$

$$\mathbf{P}^2 = T^2(\mathbf{P}^1) = T^2(T^1(\mathbf{P}))$$

$$\mathbf{P}^3 = T^3(\mathbf{P}^2) = T^3(T^2(T^1(\mathbf{P})))$$

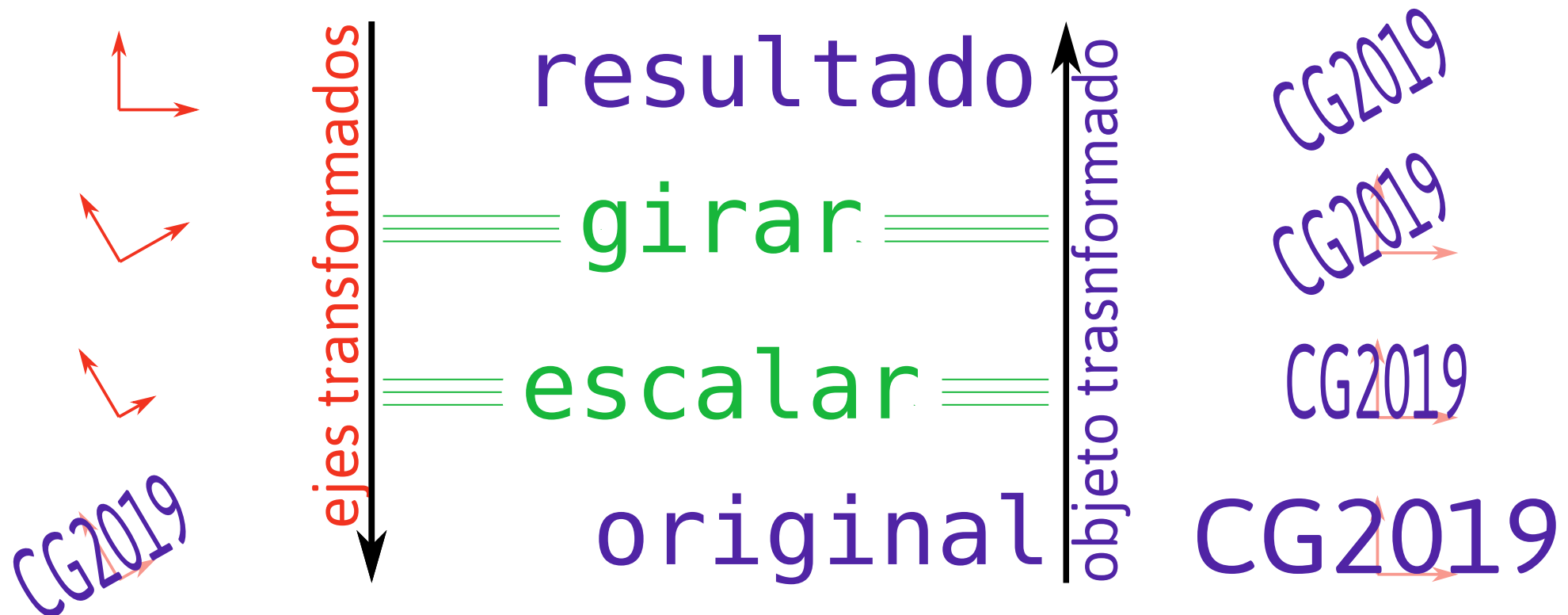
Combinación utilizando las matrices asociadas:

$$\begin{aligned}\mathbf{P}^3 &= T^3(T^2(T^1(\mathbf{P}))) = \\ &= \underline{\underline{\mathbf{M}}}^3(\underline{\underline{\mathbf{M}}}^2(\underline{\underline{\mathbf{M}}}^1\mathbf{P})) = \\ &= (\underbrace{\underline{\underline{\mathbf{M}}}^3\underline{\underline{\mathbf{M}}}^2\underline{\underline{\mathbf{M}}}^1}_{\underline{\underline{\hat{\mathbf{M}}}}})\mathbf{P} = \underline{\underline{\hat{\mathbf{M}}}}\mathbf{P}\end{aligned}$$

Si queremos aplicar por ejemplo 3 transformaciones encadenadas (tomamos $\mathbf{p}$, le aplicamos T^1 , al resultado T^2 , y al resultado T^3), es mucho más eficiente representarlas como matrices:

- Si son funciones genéricas efectivamente tenemos que hacer los 3 pasos (por cada vértice a transformar).
- Si son matrices, podemos cambiar la asociatividad (los paréntesis) y ver que podemos precalcular una sola matriz que represente las tres transformaciones juntas. La matriz se compone una sola vez, y luego por cada vértice se hace un solo producto matriz-vector, sin importar cuántas transformaciones eran originalmente.

COMPOSICIÓN DE TRANSFORMACIONES



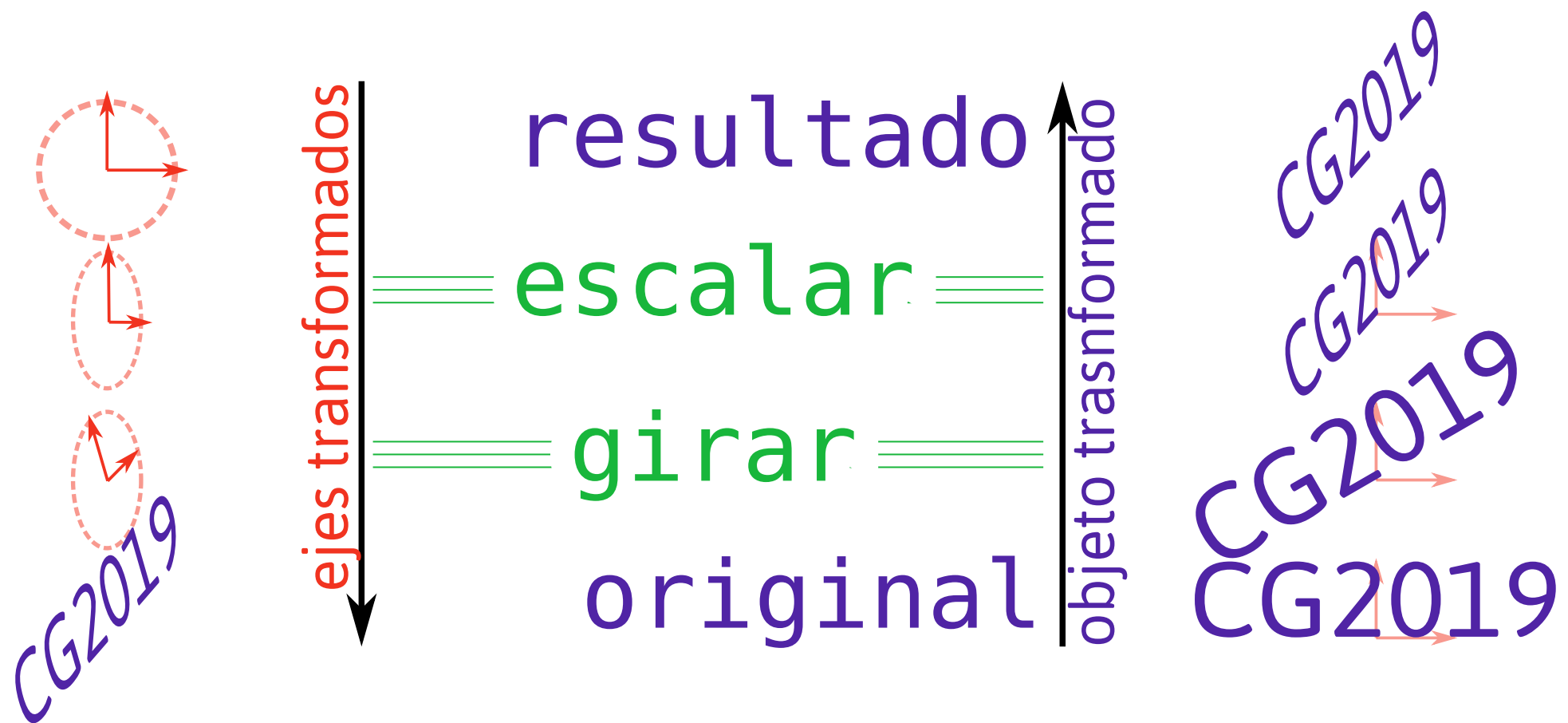
En las expresiones finales de la slide podíamos cambiar la asociatividad (los paréntesis) de los productos de matrices sin que eso altere el resultado. No aplica lo mismo respecto al orden. En matrices no es lo mismo $A*B$ que $B*A$.

Entonces resulta importante comprender el orden de las matrices/transformaciones, y hay dos formas de interpretarlo:

- Si las pensamos en el orden en que se escribe ($A*B*C*p$, A primero), tenemos que pensar que la transformación aplica al sistema de referencia: sería como ir alterando el sistema de referencia con las matrices, y "dibujar" finalmente el objeto según el sistema resultante (izquierda, de arriba hacia abajo).
- Si las pensamos en el orden que se "aplican" (como en la slide anterior, $A*B*C*p$ venía de $A(B(Cp))$), primero se aplica C), entonces tenemos que pensar que efectivamente vamos transformando el objeto (derecha, de abajo hacia arriba).

La segunda visión parece en general más intuitiva (aunque decir "primero dibujo y luego nuevo" no tiene sentido, si ya dibujé ya está!), y es además en general más fácil de pensar/entender (ej. en la próxima slide).

COMPOSICIÓN DE TRANSFORMACIONES



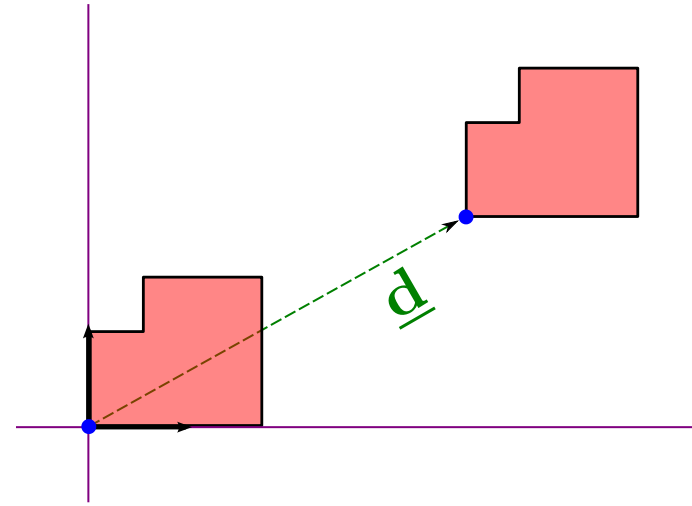
El problema de pensar en base/ejes transformados es que las transformaciones puede romper las métricas (alterar, pero de forma que ya no nos resulte fácil o intuitivo interpretar). En particular el escalado no uniforme, o el shear, de acuerdo a su posición en la lista de transformaciones, son problemáticos. Después de estas transformaciones, las bases pueden ya no medir lo mismo, o el ángulo entre ellas ya no ser 90 (dejan de ser ortonormales), y entonces es más difícil pensar, por ej, qué significa un giro de 45 grados en esa base donde 90 no es 90, o donde un círculo en realidad se convirtió en una elipse.

Por esto es que:

- Si puedo evitar tener que componer/descomponer transformaciones, mucho mejor. Aquí la interpretación de las columnas de la matriz de transformación lineal como vectores de la base transformada es clave, pues es lo que nos permite armar o entender una matriz "de una", sin andar multiplicando transformaciones más simples.
- En algunos casos igual será conveniente componer unas pocas transformaciones: en estos casos se recomienda usar el enfoque de la derecha (mover el objeto y no la base, lo cual implica "leerlas al revés") para evitar los problemas que acabamos de describir.

TRASLACIÓN

$$T(\mathbf{P}) = \mathbf{P} + \underline{\mathbf{d}}$$



$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0A + 0B \\ 0C + 0D \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

✓ Las transformaciones lineales preservan el origen

✗ La traslación no es una transformación lineal

No importa qué pongamos en la matriz, el origen (0,0) nunca se "mueve" con la transformación lineal.

- Entonces no podemos representar algo tan simple y útil como una traslación/desplazamiento: tomar cada punto p y sumarle un vector v .

- Si queremos las matrices para "ubicar" los objetos en el mundo, o en relación a la cámara, la traslación es muuuy necesaria.

Y hasta ahora no podemos codificarla dentro de la matriz de transformación (sin embargo, dado que ya sabemos que todo se implementa con matrices, tiene que haber algún "truco" para hacerlo, hacia allí estamos yendo).

TRANSFORMACIÓN AFÍN

Una transformación es **afín** si preserva la combinación afín:

$$T(\alpha \mathbf{P}_1 + \beta \mathbf{P}_2) = \alpha T(\mathbf{P}_1) + \beta T(\mathbf{P}_2) \quad \alpha + \beta = 1$$

La traslación sí es afín:

$$\begin{aligned} \alpha T(\mathbf{P}_1) + \beta T(\mathbf{P}_2) &= \alpha \overbrace{(\mathbf{P}_1 + \underline{\mathbf{d}})}^{T(\mathbf{P}_1)} + \beta \overbrace{(\mathbf{P}_2 + \underline{\mathbf{d}})}^{T(\mathbf{P}_2)} = \\ &= \alpha \mathbf{P}_1 + \alpha \underline{\mathbf{d}} + \beta \mathbf{P}_2 + \beta \underline{\mathbf{d}} = \\ &= \alpha \mathbf{P}_1 + \beta \mathbf{P}_2 + \underbrace{(\alpha + \beta)}_1 \underline{\mathbf{d}} = T(\alpha \mathbf{P}_1 + \beta \mathbf{P}_2) \end{aligned}$$

❓ Pero no puede representarse como matriz... ¿o sí?

La traslación no es lineal, pero sí afín. En la demostración de abajo partimos de transformar los puntos primero (con solo una traslación) y luego combinarlos afín/linealmente. Operando un poco, en el último renglón llegamos a que si alfa y beta suman 1 (esto es, que la combinación es afín, no solo lineal), entonces equivale a primero hacer la combinación y luego transformar.

Una transformación afín equivale a una transformación lineal más una traslación, que es todo lo que necesitamos para las matrices model y view. El problema hasta ahora es que si no es lineal no lo podemos representar como matriz.

TRANSFORMACIÓN AFÍN

Una transformación afín en $\mathbb{R}^N$ es lineal en $\mathbb{R}^{(N+1)}$:

parte lineal traslación

$$\begin{bmatrix} A & B & T_x \\ C & D & T_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} Ax + By + 1T_x \\ Cx + Dy + 1T_y \\ 0x + 0y + 1 \end{bmatrix}$$

parte fija

⚠ la dimensión adicional no es z , sino w

✅ w vale 1 para puntos y 0 para vectores

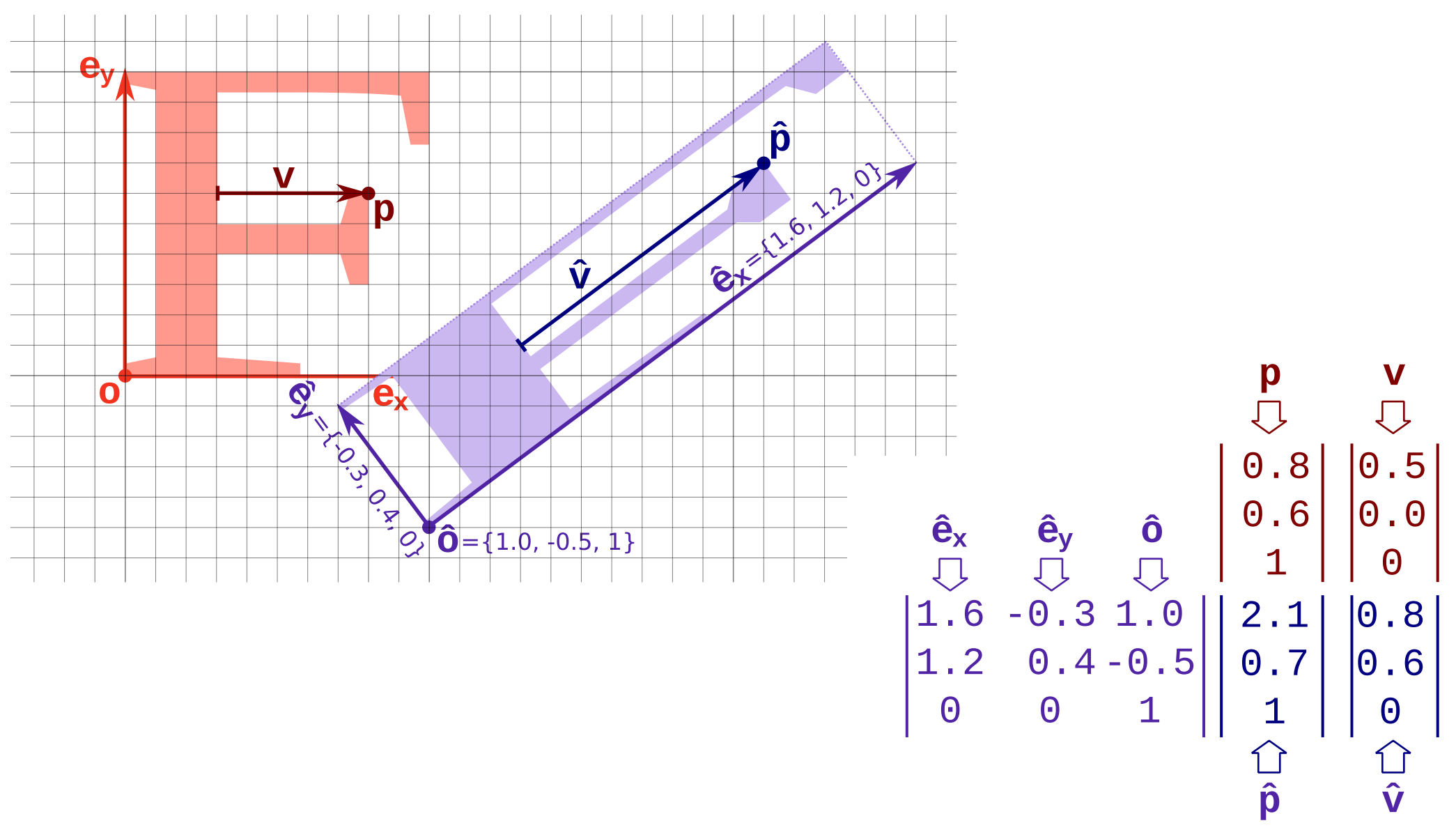
Si agregamos una dimensión más a la matriz de $N \times N$ la transformación lineal en $\mathbb{R}^N$, podemos hacer que la translación sea lineal (en ese nuevo espacio "ampliado"). Esa tercera (si partimos de $\mathbb{R}^2$) dimensión adicional no es z , sino una coordenada especial que llamaremos w .

- A este tipo de coordenadas se las llama "homogéneas", y al espacio "proyectivo".
- El espacio proyectivo P^2 es un espacio de 3 dimensiones (x,y,w) ; P^3 uno de 4 (x,y,z,w) .
- Por ahora dejaremos la fila adicional de la matriz "sin tocar" (como viene de la matriz identidad), y sí usaremos la columna adicional para lograr la translación.

Diremos que para vectores/direcciones w debe valer 0, y para puntos 1 (en realidad cualquier w diferente de 0 será un punto, pero por ahora solo es fácil de interpretar si $w=1$).

- Esto "funciona" perfectamente con las ideas que planteamos en el repaso de álgebra: que punto + vector da punto ($1+0=1$), que vector + vector da vector ($0+0=0$), que solo tiene sentido combinar puntos cuando los pesos suman 1 ($\alpha \cdot 1 + (1-\alpha) \cdot 1 = 1$), etc. En este nuevo espacio "ampliado", puntos y vectores ya no son dos tipos de cosas diferentes, solo elementos con distintos valores.

MATRIZ DE TRANSFORMACIÓN AFÍN



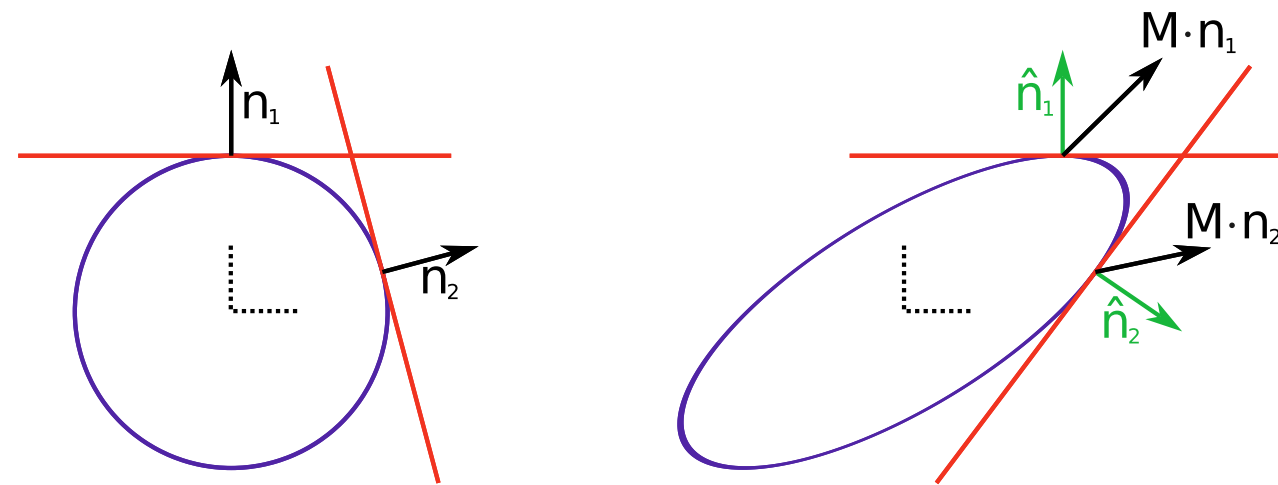
Esta es la versión ampliada (y final) del "típico" ejercicio de parcial o examen que mencionamos antes. La idea es exáctamente la misma, o bien dada la F transformada encontrar la matriz, o bien dada la matriz dibujar la F transformada. La técnica es la misma, usar la interpretación que dice que las columnas de la matriz son los vectores de la base y (esto es lo nuevo ahora) la posición del nuevo origen.

Solo se debe tener cuidado de diferenciar correctamente cuándo se trata de un vector y cuándo de un punto. En el ejemplo con números, se transforma un punto y un vector para enfatizar la diferencia. No solo uno tiene $w=1$ y el otro $w=0$, sino que al "medirlos" o "dibujarlos" se debe tener en cuenta que un vector no tiene "una" posición.

- Por ej, el vector $(2,2,0)$ se dibujará como una flecha que apunte al punto $(2,2)$ solamente si parte del origen. En el ejemplo, v no parte del origen, apunta al punto $(0.8,0.6)$, pero sus componentes son $(0.5,0)$. Lo mismo aplica al momento de medir o dibujar los vectores de la base transformada (que como ahora tienen una traslación ya no parten del $(0,0)$ original, sino del nuevo origen).

TRANSFORMACIÓN DEL VECTOR NORMAL

Las normales no se deben transformar con la misma matriz M :



✓ Pero las tangentes sí siguen siendo tangentes

$$\begin{aligned} \underbrace{\mathbf{n} \cdot \mathbf{t} = 0}_{\mathbf{n}^T \mathbf{t}} &= \underbrace{\mathbf{n} \cdot \mathbf{t} = 0}_{\mathbf{t}} = \underbrace{\hat{\mathbf{n}} \cdot \hat{\mathbf{t}} = 0}_{\hat{\mathbf{n}}} = \underbrace{\hat{\mathbf{n}} \cdot \hat{\mathbf{t}} = 0}_{\hat{\mathbf{n}} = (\mathbf{M}^{-1})^T \mathbf{n}} = ((\mathbf{M}^{-1})^T \mathbf{n})^T \hat{\mathbf{t}} \end{aligned}$$

Para transformar normales se usa la transpuesta de la inversa de M .

Supongamos que transformo un objeto aplicándole una matriz de transformación M a cada vértice. No puedo usar esa misma matriz para transformar las normales. Como se ve en la figura, las normales transformadas de esta forma ya no son normales (ya no son ortogonales a los vectores/rectas/planos tangentes).

- El vector se define en base a la ortogonalidad con las tangentes. Pero si la transformación altera las métricas (recordar el ejemplo del orden en la composición), entonces 90 grados transformados ya no son 90 grados desde el punto de vista del sistema original.

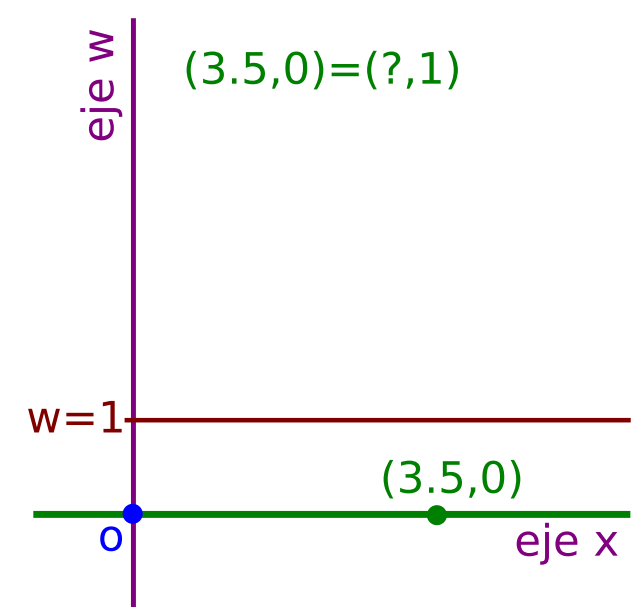
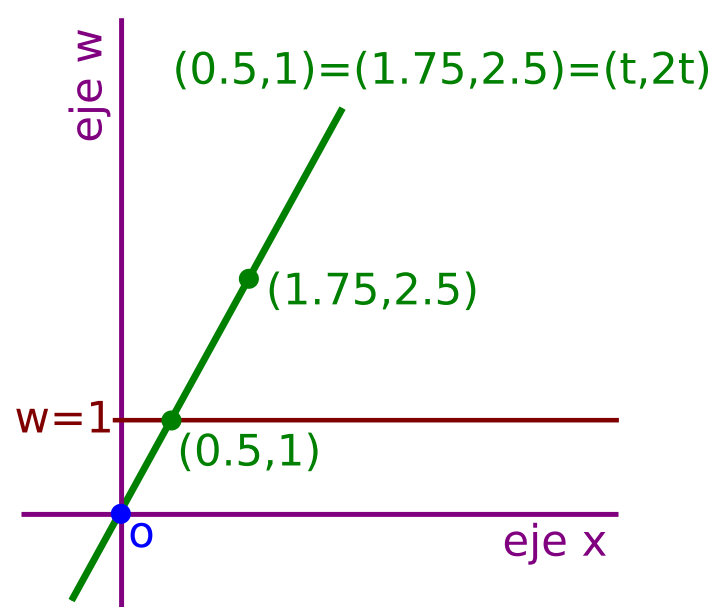
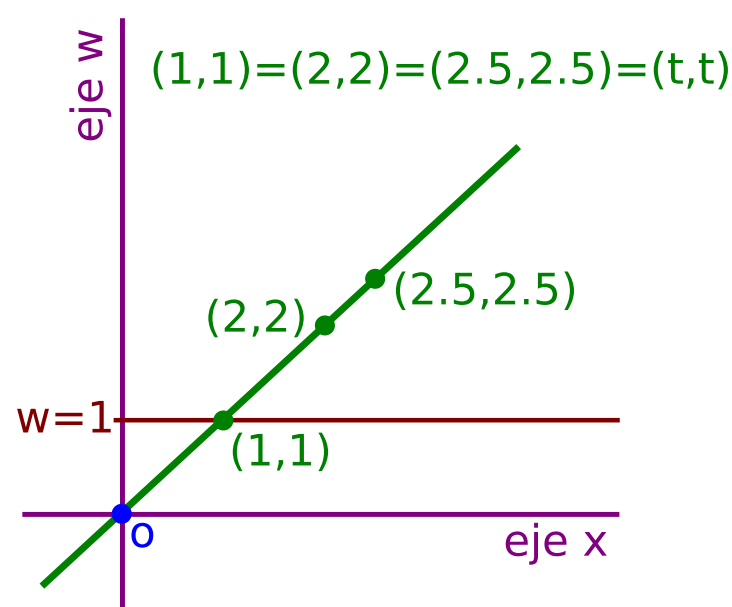
Pero una recta tangente sí se transforma "correctamente" con esa matriz.

- Pensemos que un vector tangente se define en base al límite de la diferencia entre dos puntos de la curva o superficie, cuando los puntos tienden a ser el mismo (uno tiende al otro). Como estamos hablando de una resta de puntos, y los puntos sí se transforman correctamente, entonces la misma "resta" en la versión transformada va dar la tangente correcta.

Podemos usar las tangentes (original y transformada) para deducir la matriz de transformación que nos de las normales correctas. Esa matriz resulta ser la transpuesta de la inversa de la matriz original M . No es importante recordar esta demostración, pero sí el hecho de que hay que tratar a las normales de forma especial (veremos por ejemplo en todos los vertex-shaders que las normales por vértice que se requieren para iluminación se transforman de esta manera).

ESPACIO PROYECTIVO

- P^N tiene una dimensión/coordenada(w) más que R^N
- Los elementos de este espacio son rectas que pasan por el origen.
 - Quedan definidas con un solo punto (x,y,w) .
- Los identificamos con el plano $w=1$
 - Coordenadas homogéneas: $(x,y,w) \rightarrow (x/w, y/w, 1)$



Generalizando las ideas del espacio proyectivo:

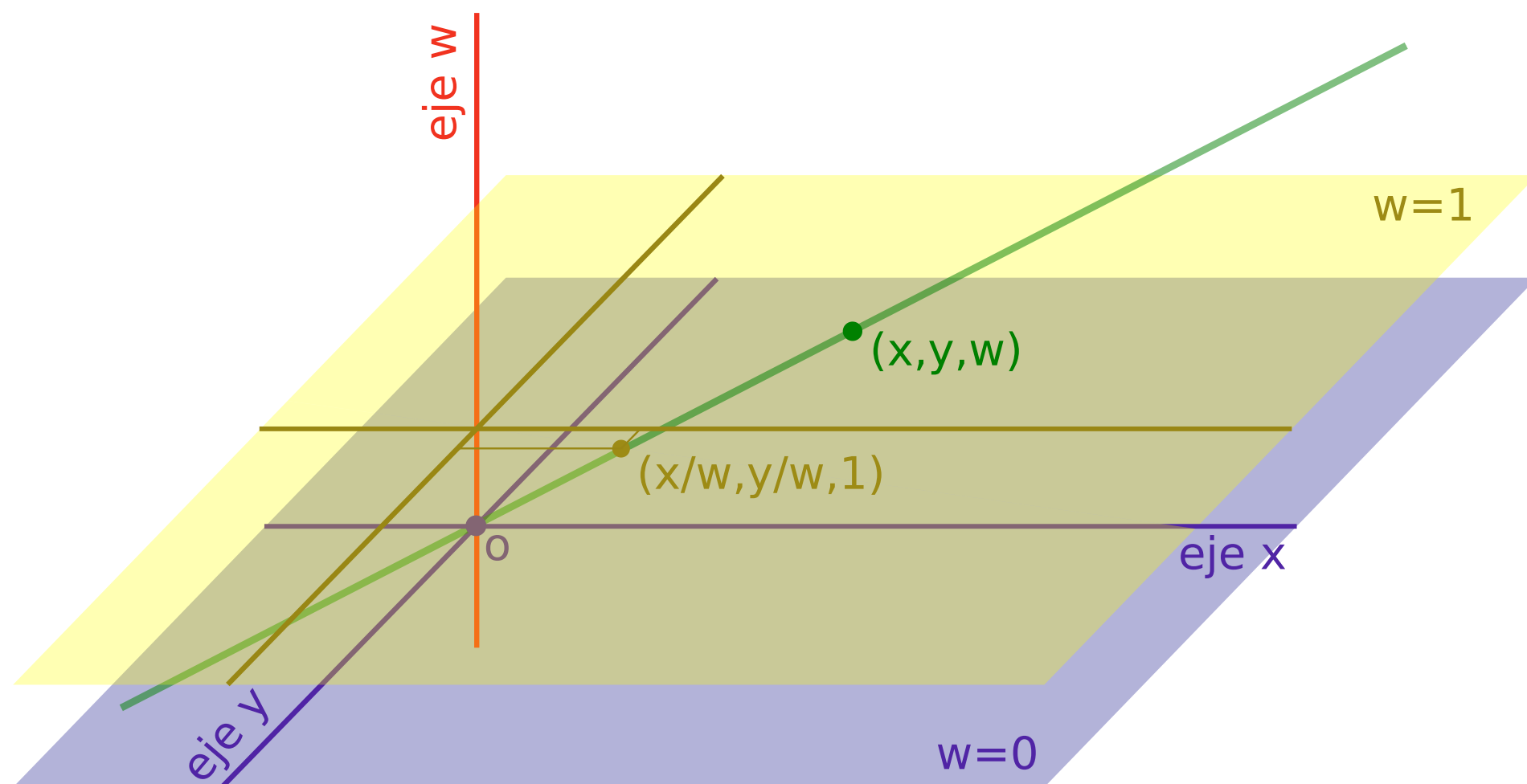
- Siempre parto de R^N y agrego w , entonces termino trabajando en $(N+1)$ dimensiones, y a ese espacio se le llama P^N .

- Los "elementos" de este espacio vectorial son rectas y no puntos. Y son rectas que pasan por el origen, por lo que puedo definirlos con un solo "punto" (porque el otro es el origen), por eso cada "elemento" parece las coordenadas de un punto, pero en realidad identifica una recta. En las figuras estamos dibujando P^1 como si fuera R^2 . Los puntos $(1,1)$, $(2,2)$, $(2.5,2.5)$ definen con el origen la misma recta, para el espacio proyectivo serían equivalentes.

- Nos interesa lo que pasa sobre el "plano" $w=1$ ("plano" en general, la "recta" en estas figuras). Si tenemos un punto en R^N y lo queremos pasar a P^N definimos su w como 1. Lo ubicamos "sobre" el plano; aunque representa una recta, nos interesa especialmente lo que pasa donde esa recta se intersecta con el plano $w=1$, porque esas son las coordenadas que vamos a tomar cuando debamos "volver" a R^N (por eso "dividimos por w " para "volver").

Esta idea de que nos interesa dónde cada recta de este espacio corta a $w=1$ no funciona para vectores (ya que son paralelos a ese plano), ¿o sí?

ESPACIO PROYECTIVO

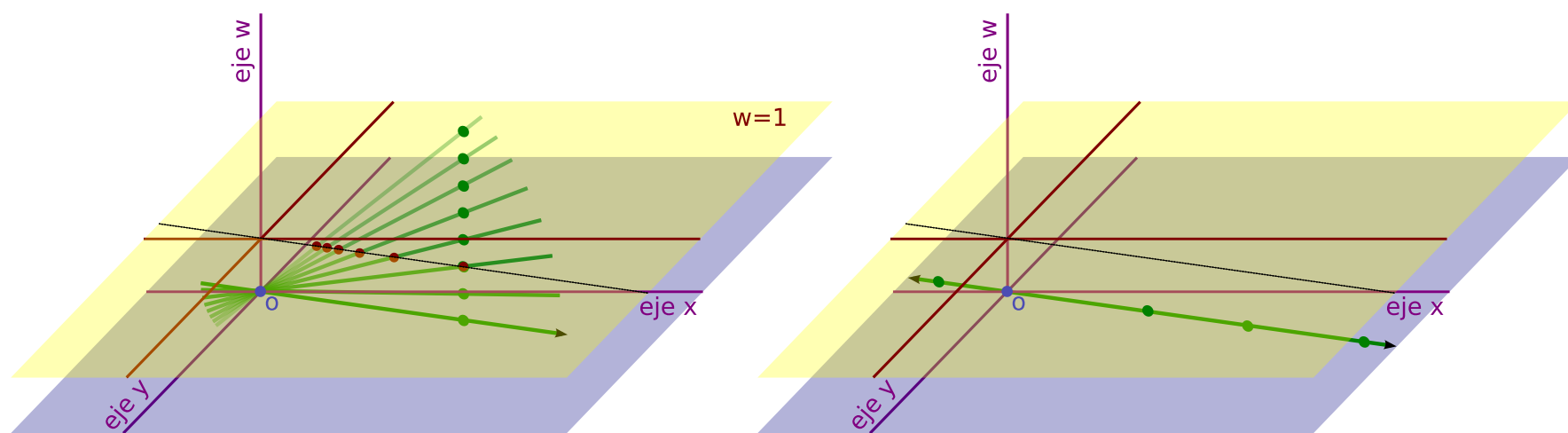


Aquí hay un ejemplo en P^2 (x, y, w) . El plano azul es $w=0$, el amarillo $w=1$. El punto verde (x, y, w) define la recta verde, y esa recta corta a $w=1$ en $(x/w, y/w, 1)$.

Para la mayoría de los ejemplos utilizaremos gráficos de P^2 , porque se puede asimilar a R^3 (tiene 3 dimensiones) y de esta forma dibujarlo. En el pipeline en realidad trabajamos en P^3 , pero no podríamos dibujar algo de 4 dimensiones (no sin proyectar 2 veces, y eso sería muy poco intuitivo).

ESPACIO PROYECTIVO Y PLANO IDEAL

- Si $w=0$
 - la recta es horizontal
 - se corresponde con un punto ideal, en el infinito
 - definido por un "vector" dirección



El plano $w=0$ es el "plano ideal"

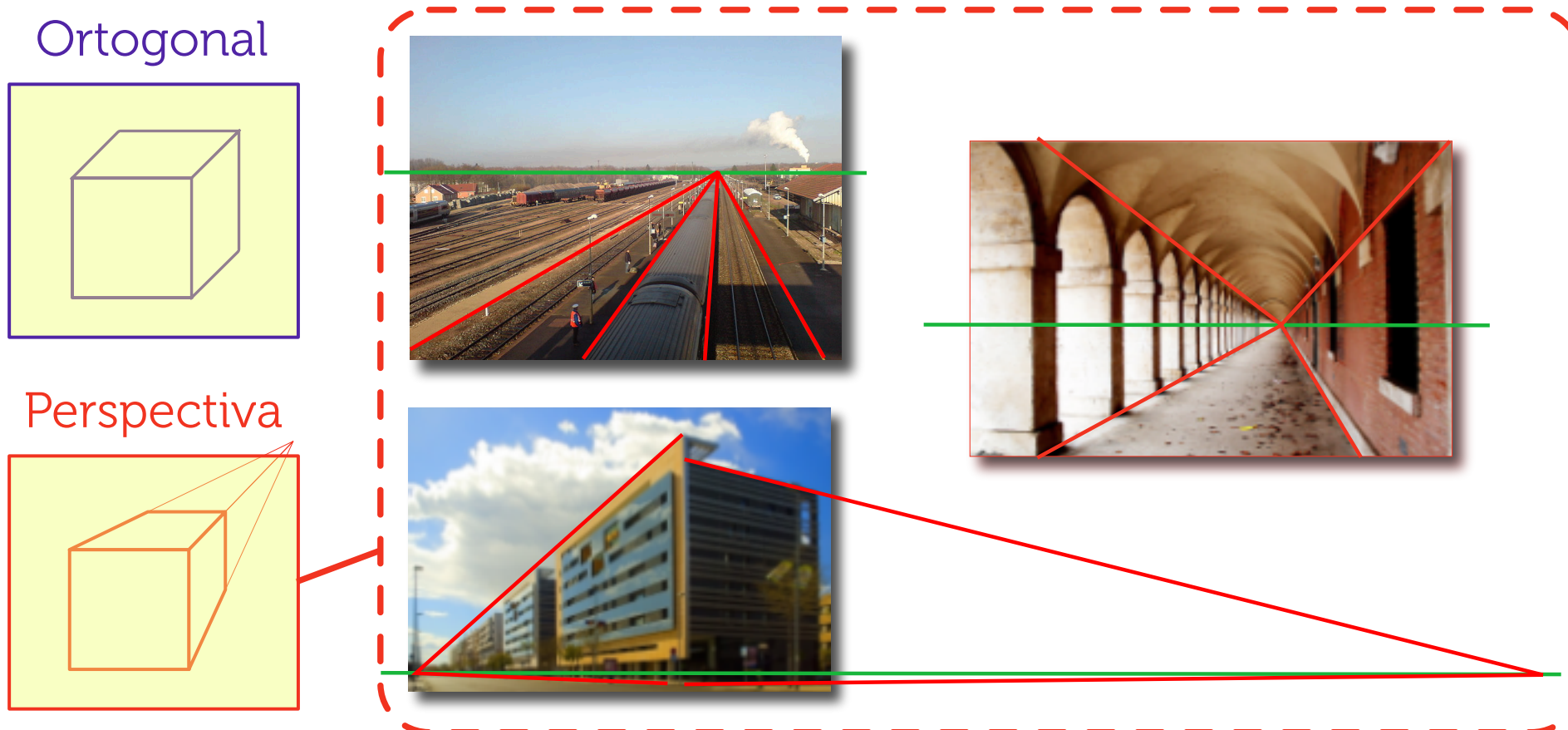
Para entender qué representan los "puntos" sobre el plano $w=0$ (al que se le llama plano "ideal"), partamos de un punto con $w \neq 0$, y hagamos tender su w a 0. En la figura de la izquierda, estos puntos son los puntos verdes: a medida que w se acerca a 0 (el punto verde baja), y la intersección con el plano $w=1$ se aleja en cierta dirección (se mueve sobre una recta). A medida que w tiende a 0, la intersección tiende al infinito en esa dirección.

Los "puntos" con $w=0$ representan puntos en el infinito (o mejor su dirección). El plano ideal representa el infinito.

Notar que hay infinitos infinitos: el infinito tiene dirección, por cada par de direcciones opuestas hay un "infinito" (una recta en este plano ideal). Y notar lo de "pares" de infinitos opuestos: si el punto sobre $w=1$ tiene al infinito en dos direcciones opuestas, en ambos casos tiende a la misma recta sobre el plano ideal (son equivalente en el espacio proyectivo).

PROYECCIONES

Una proyección es una transf. de rango incompleto, no invertible.



⚠ *Una transformación proyectiva no es una proyección*

Por el nombre, puede generarse confusión entre "proyección" y "transformación proyectiva".

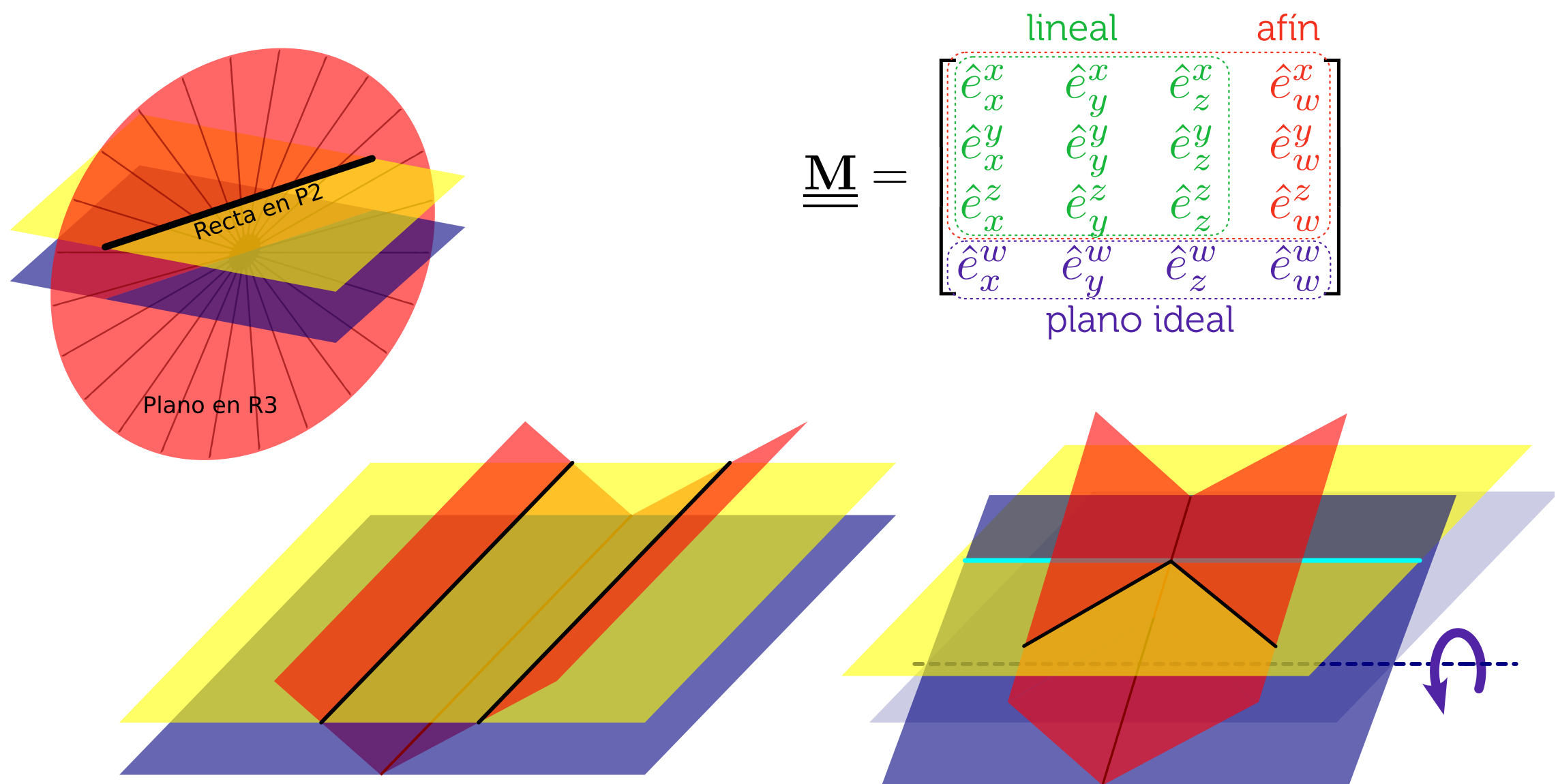
Una transformación proyectiva en P^N es como una transformación lineal en $R^{(N+1)}$. Entonces, en general no es una proyección (solo lo serán algunos casos particulares).

Vamos a querer proyecciones para pasar de nuestro mundo virtual 3D al plano de la imagen 2D.

- La **proyección ortogonal** simplemente "aplata": digamos que descarta la profundidad, z en el espacio del ojo, y se queda con x, y inalterados. No es real, pero es útil para cosas como CAD, ya que hace más fácil comparar cosas como ángulos y proporciones.

- La **proyección perspectiva** es la que "vemos" en la realidad, donde las cosas que están más lejos se ven más pequeñas. Esto quiere decir que, aún en el espacio del ojo, los valores finales de x, y en el plano, dependerán de z . Por esto no es una transformación lineal: los coeficientes de la matriz deben ser constantes, no pueden ser funciones de z . Pero veremos que al igual que pasó con la traslación, el espacio proyectivo nos ayuda a hacer algún truco para codificar esta proyección dentro de la matriz igualmente.

TRANSFORMACIONES PROYECTIVAS



Notar que las infinitas rectas en P^2 que corresponden a los infinos puntos de una recta en R^2 , forman un plano (dos puntos cualquiera de la recta y el origen lo definen). Y dos rectas paralelas en R^2 son dos planos en P^2 que se cortan en $w=0$. Por cada dirección en R^2 , hay una recta en $w=0$ donde se cortan todas las paralelas en esa dirección. Esto concuerda con que $w=0$ representa a las cosas en el infinito.

Si giro el plano $w=0$, ahora se intersecta con el $w=1$ original. Esto es, desde el punto de vista del $w=1$ original, puntos que estaban en el infinito, ahora están en lugar concreto (son los puntos de fuga, quedan sobre la línea del horizonte).

Sobre la interpretación de la matriz:

- Si parto de la identidad y solo toco la parte verde (los vectores base de R^N transformados) tengo una transformación lineal.
- Si además modifico la parte roja (el nuevo origen) tengo una transformación afín (lineal+traslación).
- Si modifico la parte azul (especialmente los 3 primeros) estoy cambiando el plano ideal, tengo una transformación proyectiva propiamente dicha.

Entonces, el uso del espacio proyectivo nos permite unificar puntos y vectores en una mismo tipo de elemento, implementar fácilmente (mediante matrices) cosas que no serían lineales (traslación, proyección perspectiva), y nos permite representar sin problemas (sin valores especiales con $\#inf$ o $\#NaN$) puntos en el infinito. Y todo lo que podamos poner en matrices será muy eficiente al momento de la implementación (el código y el hardware).

Por esto si bien parece una idea en principio difícil y poco intuitiva, es una solución muy muy elegante a muchos problemas complejos. Uno puede aprender las recetas y entender poco de lo que pasa detrás, y arreglárselas igual para hacer cosas en 3D. Pero se espera que un ingeniero tenga la capacidad de entender mejor las herramientas que está usando y sus fundamentos.

¿QUÉ HAY QUE SABER?

- Transformaciones en el pipeline
 - Propósito de las 3 matrices (model, view, projection)
 - Por qué usamos matrices
 - Principales espacios: modelo, mundo, ojo, NDC
- Transformación **lineal y afín**
 - Concepto, utilidad
 - **Armar la matriz (sin componer)**
 - **Interpretar una matriz dada (sin descomponer)**
- Espacio Proyectivo/Coordenadas Homogéneas
 - Concepto
 - Objetivos/ventajas